

第7章 大模型智能体的原理与实践

基于聊天的大模型能写出流利的实验步骤，却无法替你点击鼠标；它能生成完美的代码，却不能自己按下回车。这种“知”与“行”的割裂，让所有关于人工智能的设想，始终停留在对话框的彼岸。智能体的出现，正是为了打破这面玻璃墙。它不再满足于做一位只动口不动手的咨询助理，而是要成为能亲自为你干活的个人助理。从“思考”到“行动”，从“建议”到“执行”，智能体赋予了大模型真正的行为能力：调用工具、操作软件、甚至指挥机械臂。本章将带领大家完成这场从“旁观”到“介入”的认知升级，系统拆解智能体的核心架构、记忆机制、工具调用与 workflow 编排，并最终通过实战案例，让你亲手构建属于自己的智能体。

7.1 智能体的基础

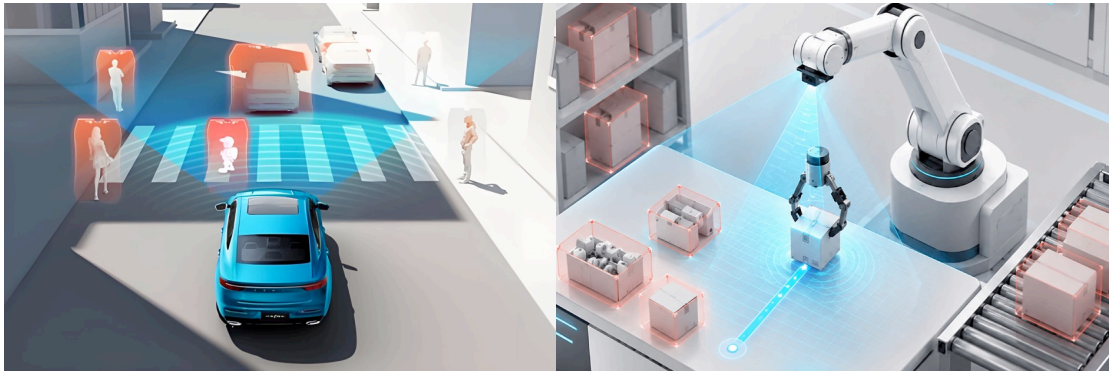
智能体是大模型从“内容生成工具”走向“任务执行系统”的重要形态。与传统问答式大模型不同，智能体不仅能够理解用户需求，还能够围绕具体目标进行任务拆解、工具调用、过程管理和结果反馈。它使大模型不再只是被动回答问题，而是具备了一定的感知、规划、行动和记忆能力。本节将从智能体的基本概念出发，进一步分析其核心能力并概述当前主流的架构类型与分类范式。

7.1.1 什么是智能体

智能体是一种能够围绕特定目标感知外部信息、理解任务要求，并据此制定计划和采取行动的智能系统。在人工智能领域，智能体并不是一个新概念。日常生活中常见的智能体如图 7-1 所示，包括自动驾驶系统和智能机器人，是两类典型的智能体形态。自动驾驶系统通过感知道路环境和车辆状态，完成路线判断、速度调节与制动控制；智能机器人则依靠传感器获取外部信息，并根据任务目标规划运动路径，完成抓取或搬运等操作。虽然这些系统的应用场景和实现方式并不相同，但其基本运行逻辑具有一致性，即先从环境中获取信息，再进行分析判断，最后通过具体行动作用于外部环境。

随着大语言模型的发展，智能体的内涵逐渐从传统的专用智能系统，扩展为面向开放任务的通用智能系统。传统智能体通常服务于边界清晰且目标明确的应用场景，其行为更多依赖预设规则、专用模型或固定流程。基于大语言模型的智能体则具有更强的语言理解能力和任务组织能力，能够根据自然语言描述识别任务目标，并进一步分析任务所需的信息与执行

步骤。智能体则负责理解用户的需求，并将其转化为可以逐步执行的操作过程。由此，智能体从面向特定场景的自动化系统，进一步发展为支撑复杂任务处理的通用执行框架。



(a) 自动驾驶系统 (b) 智能机器人

图 7-1 日常生活中常见的智能体

在大模型智能体中，LLM 通常承担“脑”的功能。它并不直接完成外部操作，而是负责理解用户意图，分析任务目标，并将复杂问题转化为较为清晰的执行思路。例如，当用户希望系统整理一份关于“人工智能教育应用”的报告时，LLM 首先需要判断这一任务的核心要求，明确报告应围绕哪些内容展开，再据此形成相应的写作框架和处理步骤。由此可见，LLM 为智能体提供了任务理解、知识推理和过程规划的基础能力，是智能体进行判断与决策的核心部分。

然而，仅具备“脑”的能力，并不足以支撑智能体完成真实任务。大语言模型擅长语言理解和文本生成，但其自身并不能直接访问实时数据，也不能主动读取本地文件、运行程序或操作外部设备。当任务涉及数据统计、文献检索、实验计算或业务系统操作时，仅依靠模型内部知识和语言生成能力，往往难以满足实际需求。此时，智能体需要借助外部工具，将推理结果进一步落实为具体操作。

因此，工具系统可以被视为智能体的“手”。它使智能体能够连接外部环境，获取必要的信息，执行相应的计算或操作，并将执行结果返回给模型进行后续判断。例如，在科研和办公场景中，智能体可以通过检索工具获取资料，通过计算工具处理数据，通过文档工具生成报告，也可以通过接口与业务系统进行交互。借助这些外部能力，大语言模型不再局限于文本层面的响应，而能够进一步参与信息获取、数据处理和任务执行的完整过程。

智能体的“脑”与“手”如图 7-2 所示，在实际运行中，大模型智能体通常以循环方式推进任务。用户提出需求后，系统首先理解任务内容，判断当前问题需要依赖哪些信息来源或外部能力。随后，工具系统根据模型给出的指令完成相应操作，并将结果返回给模型。模型再依据反馈判断任务是否已经完成，或者是否需要进一步调整思路。通过这样的循环，智

智能体能够在持续获得新信息的过程中修正判断，使任务逐步向目标推进，而不是停留在一次性的文本回应上。

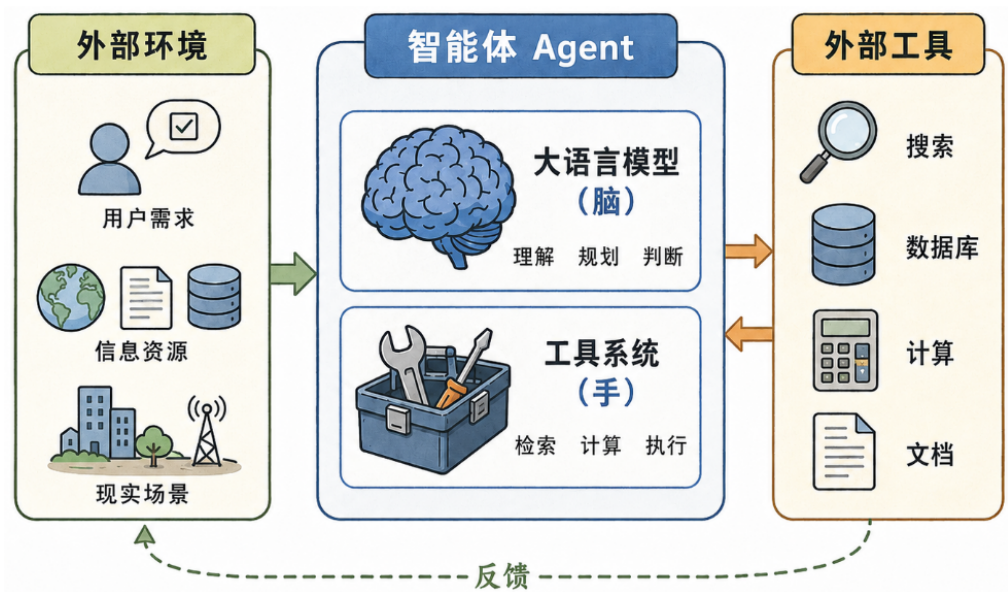


图 7-2 智能体的“脑”与“手”

7.1.2 智能体的核心能力

如果把上一节介绍的智能体比作一位能够自主完成任务的助手，那么本节要回答的问题就是：这位助手究竟需要具备哪些基本能力，才能真正从“回答问题”走向“完成任务”？一般来说，一个完整的智能体系统通常离不开四类核心能力：感知、规划、行动与记忆。它们并不是彼此孤立的模块，而是共同构成了智能体持续运行的任务闭环。

感知能力使智能体能够接收和理解外部输入，规划能力帮助智能体将复杂目标拆解为可执行步骤，行动能力使其能够调用工具并完成实际操作，记忆能力则保证智能体能够保存任务过程、历史信息和用户偏好。只有当这四种能力协同工作时，智能体才能不再局限于一次性的文本回答，而是具备持续处理任务的能力。

智能体核心能力协同机制如图 7-3 所示。该图以大模型为中心，展示了感知、规划、行动和记忆如何共同形成动态闭环：感知负责输入信息，规划负责制定步骤，行动负责执行任务，记忆负责保存经验，而反馈结果又会重新进入下一轮判断与执行过程。

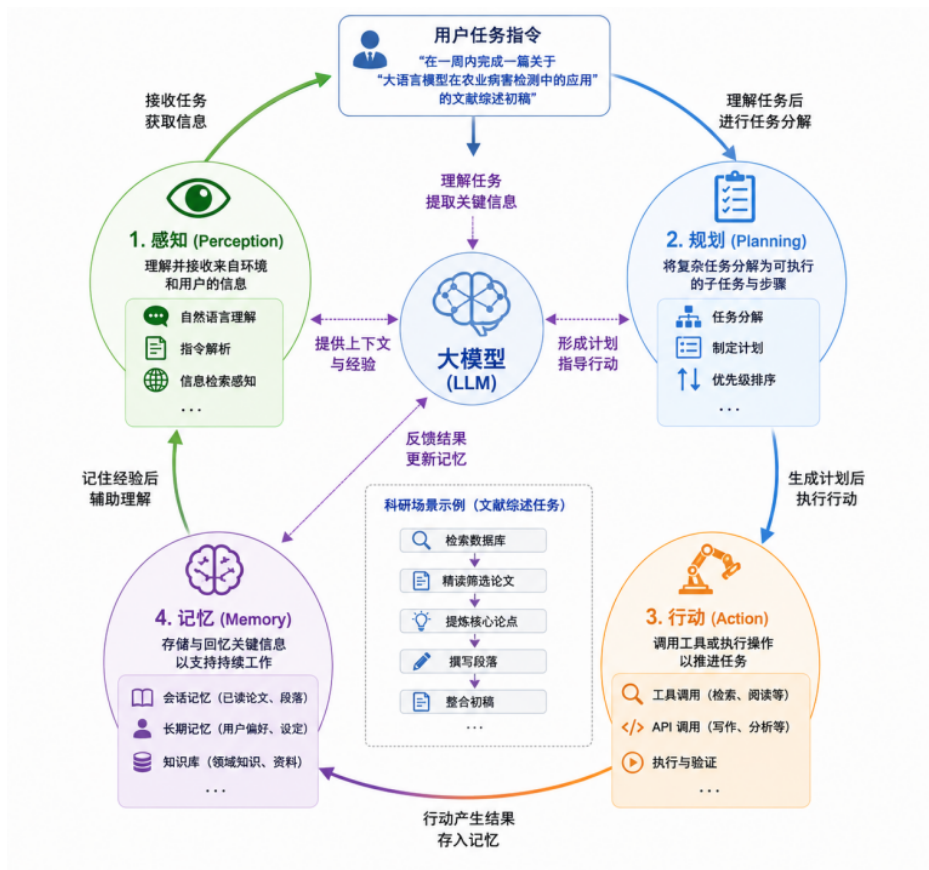


图 7-3 智能体核心能力协同机制

1. 感知：智能体的信息入口

感知是智能体与外部世界之间的第一道桥梁，也是整个任务闭环的起点。它决定了智能体能够接收哪些信息、理解哪些内容，并进一步影响后续规划和行动的质量。在大模型智能体中，感知能力主要依赖模型的输入通道和外部工具返回的信息。目前常见的感知输入包括以下几类：

第一类是文本信息，如用户指令、对话历史、网页内容、代码片段和文档内容等。这是当前最基础、最成熟的感知形式。大多数智能体任务都从文本输入开始，例如用户提出一个问题、上传一段资料，或者描述一个希望完成的目标。

第二类是结构化数据，如表格、数据库查询结果、API 返回的数据等。这类数据通常具有明确字段和规则，智能体需要理解其中的数值、标签和关系，才能进一步完成统计、查询或分析任务。

第三类是图像与多模态信息。随着视觉语言模型的发展，智能体已经能够解析图片、截图、图表甚至部分视频内容。例如，在处理实验图像、网页截图或界面操作任务时，图像信息就可能成为重要的感知来源。

第四类是工具执行反馈。智能体调用搜索、计算、文件处理等工具后，工具返回结果也会成为新的输入。智能体需要根据这些反馈判断任务是否完成，或者是否需要继续调整策略。

由此可见，感知能力的强弱，直接决定了智能体能够理解任务的范围。如果输入信息不完整，或者系统无法正确解析外部反馈，那么后续的规划和行动也容易出现偏差。因此，在构建智能体时，首先要明确它需要接收什么信息，以及这些信息如何被组织和传递给模型。

2. 规划：智能体的任务组织能力

如果说感知解决的是“获取什么信息”，那么规划解决的就是“接下来怎么做”。规划能力的核心作用，是将用户提出的复杂目标拆解为一系列可以执行的步骤，并根据任务进展不断调整执行策略。

对于简单问题，大模型可以直接生成回答；但对于复杂任务，仅靠一次生成往往难以保证结果质量。例如，完成一份资料整理任务，通常需要经历明确主题、检索资料、筛选内容、归纳观点、组织结构和生成结果等多个环节。智能体的价值就在于，它能够把这类复杂目标转化为分阶段、可执行的任务流程。

在智能体中，常见的规划方式是“思考—行动—观察”的循环机制。系统先根据当前任务进行分析，判断下一步需要调用什么工具或执行什么操作；随后执行该操作，并接收返回结果；最后再根据观察结果决定是否继续执行、修改计划或结束任务。这种机制使智能体不再只是一次性生成答案，而是能够围绕任务目标持续推进。

需要注意的是，规划并不是一次性完成的固定方案。实际任务中经常会出现信息不足、检索结果不准确、工具调用失败或生成结果偏离目标等情况。此时，智能体需要根据反馈重新判断任务状态，并对原有计划进行修正。正是这种动态调整能力，使智能体区别于普通的自动化脚本。

3. 行动：智能体的实际执行能力

行动能力是智能体区别于普通大模型的关键。普通大模型主要通过文本进行回应，而智能体不仅能够“说”，还能够借助外部工具“做”。它可以调用检索工具获取资料，调用计算工具处理数据，调用文档工具生成文件，也可以通过接口与外部系统进行交互。

在实际应用中，智能体常见的行动方式包括信息检索、代码执行、文件读写、外部服务调用和界面操作等。不同任务对行动能力的要求并不相同。例如，资料问答类任务更依赖知识库检索，数据分析类任务更依赖代码执行和表格处理，文档生成类任务则更依赖文件读写和格式处理。

行动能力的引入，使智能体从“语言生成系统”进一步变成“任务执行系统”。但是，

行动能力越强，风险也越高。如果智能体可以删除文件、发送邮件、修改数据或调用外部系统，那么错误操作就可能造成实际影响。因此，在设计智能体时，通常需要设置必要的权限边界和确认机制。对于高风险操作，应先向用户展示执行方案，并获得确认后再执行。

4. 记忆：智能体的持续任务支撑

记忆能力解决的是智能体如何在较长任务中保持连续性的问题。没有记忆的智能体只能依赖当前输入进行判断，容易反复询问相同信息，也难以利用历史任务经验。对于需要多轮交互、长期积累或个性化服务的任务而言，记忆能力尤为重要。

智能体的记忆通常可以分为几个层次。首先是工作记忆，即当前上下文中保存的信息，包括用户最新指令、已有对话、工具调用结果和当前任务状态。工作记忆直接影响模型当前一轮的判断，但容量有限，任务过长时容易丢失早期信息。

其次是情节记忆，主要记录智能体完成过的具体任务过程。例如，某次任务中检索了哪些资料、执行了哪些步骤、生成了什么结果。这类记忆有助于智能体在相似任务中复用经验，减少重复劳动。

再次是语义记忆，它存储的是经过归纳后的知识、规则和偏好。例如，某类任务的固定处理流程、某个领域的常用术语，或者用户长期偏好的回答格式。相比情节记忆，语义记忆更加抽象，也更适合指导后续任务。

最后是外部记忆，通常依托知识库、数据库或向量数据库实现。它突破了大模型上下文窗口的限制，使智能体能够从大量文档和资料中按需检索相关内容。对于专业领域应用而言，外部记忆往往是提升智能体可靠性和专业性的关键支撑。

总体来看，感知、规划、行动和记忆共同决定了智能体的能力边界。感知能力决定智能体理解什么，规划能力决定它如何组织任务，行动能力决定它能实际完成什么，记忆能力则决定它能否在持续交互中保持连贯和积累经验。对于不同应用场景而言，这四种能力的重要性并不完全相同。构建智能体时，需要根据具体任务目标，确定哪些能力是核心，哪些能力可以简化，从而形成既可用又可控的智能体系统。

7.1.3 智能体的主流架构与分类范式

智能体架构设计存在的差异，会影响到智能体的自主能力、执行水平和适用范围。本节将从能力演变和类型划分这两个方面，介绍智能体的主流架构和分类方法。

1. 智能体从对话能力到任务能力的演进

大语言模型的出现，使机器具备了较强的语言理解与内容生成能力。它可以回答问题、

解释概念、润色文字，并且可以根据提示生成论文提纲、实验说明或代码片段。但是，在实际科研工作中，大部分任务无法通过一次问答来完成。比如，进行文献综述写作时，一般需要经历确定主题、检索文献、筛选资料、提炼观点、比较方法、形成模式、修改表达等步骤。仅凭一次对话就生成结果，很难保证任务完成的连续性、准确性和可控性。

智能体正是在这一背景下发展起来的。与普通大模型相比，它不仅生成一段回答，还可以围绕特定目标来组织行动。它能够理解用户意图，对任务步骤进行拆解，调用外部工具，运用已有知识，并且依据执行结果对后续操作进行调整。智能体主要的变化在于它将大模型的语言能力进行转化，使其成为执行任务的能力。

智能体从对话能力到任务能力的演进如图 7-4 所示。普通大模型主要进行语言理解和文本生成；工具增强型智能体能够利用外部资源，来进行检索、计算和绘图等操作；规划执行型智能体可以按照复杂目标来完成任务的闭环。经过该演变过程，人工智能已不只负责给出回答的对话程序，而是慢慢发展成可以参加研究步骤的互助程序。

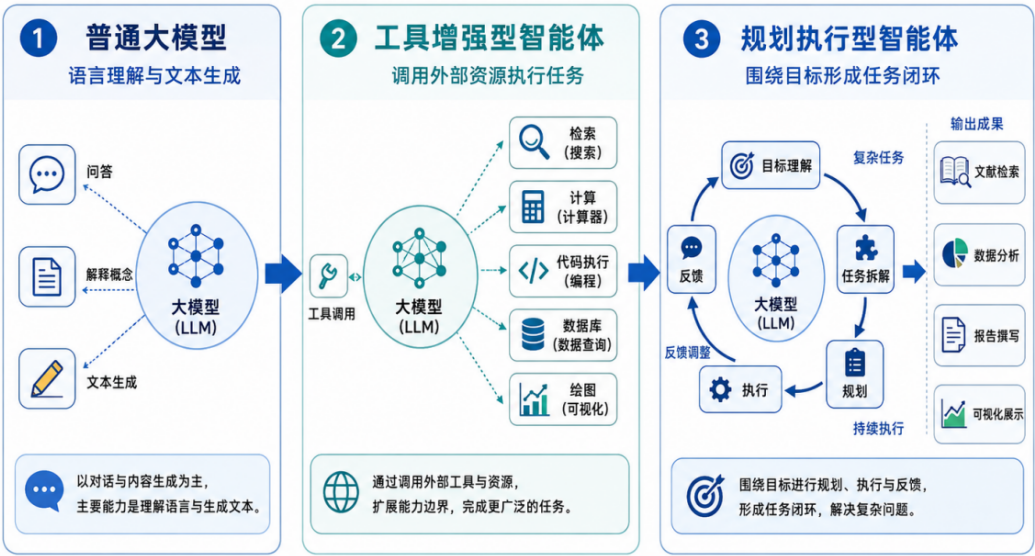


图 7-4 智能体从对话能力到任务能力的演进

2. 智能体的主流分类范式

根据能力侧重点和应用场景的差异，智能体大致可以分为以下几类。

第一类是**问答型智能体**。这是较为基础的智能体形式，主要根据用户输入完成解释、回答和文本生成，适用于概念学习、文字润色、研究思路启发等轻量化任务。其优点是使用方便、响应速度快，但通常缺乏长期记忆和复杂任务规划能力。

第二类是**知识增强型智能体**。这类智能体通常会接入专业知识库、本地文档或论文资料，在回答前先检索相关内容，再结合检索结果生成回复。它适用于论文问答、课题组资料查询和专业教材辅助学习等场景。相比完全依赖模型内部知识的方式，知识增强型智能体能够在

一定程度上减少幻觉问题，使回答更具依据性和专业性。

第三类是**工具调用型智能体**。其主要特点是能够连接并调用外部工具，完成计算、检索、绘图、代码运行和文件处理等操作，适合数据分析、代码调试、图表生成和文献检索等科研任务。对于需要实际操作的场景而言，这类智能体比单纯的文本生成系统更具有应用价值。

第四类是**规划执行型智能体**。这类智能体能够围绕复杂目标进行任务拆解，按照一定步骤逐步推进，并根据中间结果调整后续计划。它适用于开题报告辅助、文献综述写作、实验方案设计和论文修改迭代等复杂任务。不过，规划执行型智能体并不意味着可以完全替代研究者。任务越复杂，越需要人工对事实依据、数据结果、逻辑关系和学术规范进行审核。

表 7-1 对几类主流智能体的核心特征、适用场景和主要局限进行了概括。

表 7-1 主流智能体分类范式

智能体类型	核心特征	主要局限
问答型智能体	根据用户输入进行解释、回答和文本生成	缺少长期记忆和复杂规划能力
知识增强型智能体	接入文档或知识库，基于检索资料生成回答	依赖知识库质量，资料不全会影响回答
工具调用型智能体	能调用外部工具完成计算、检索、绘图等操作	执行效果受工具质量和调用准确性影响
规划执行型智能体	能拆解复杂任务，并进行分步执行与反馈修正	系统复杂度较高，需要人工监督与审核

整体上看，问答型智能体发展到规划执行型智能体，系统能力由语言生成扩大到任务执行。但并不是架构越复杂就越好。对简单概念进行解释时，可以使用问答型智能体，整理专业资料时，更适合使用知识增强型智能体，处理数据和生成图表时，需要工具调用型智能体，而进行连续性的研究任务时，规划执行型智能体更加适用。

7.2 智能体之记忆：专业知识库的构建与管理

你是否遇到过这样的场景：向大语言模型询问一篇最新发表的学术论文，它煞有介事地列出标题、作者和摘要，但当你去检索时，却发现这篇文章从未存在过；又或者，你让它解释某个小众领域的专业术语，它给出的答案初看逻辑通顺，细究却张冠李戴，混淆了关键概念。这是大模型幻觉的根源，本小节将介绍如果借助外部的专业知识库来减少大模型幻觉。

7.2.1 为什么需要专业知识库？

在使用大语言模型时，大家经常会遇到这样的情况：模型能够生成语言流畅、结构完整的回答，但其中引用的资料、数据或概念解释可能并不准确。对于一般对话影响不大，但在学习辅导、专业问答、科研辅助和业务咨询等场景中，错误信息会直接影响用户判断。因此，智能体在专业任务中，需要依赖可靠的知识来源，而不能完全依赖模型内部知识。

大模型幻觉的两个例子回答如图 7-5 所示，当一个问题需要调用的信息在训练数据中稀疏、矛盾或者压根不存在时，模型不会像人类一样坦承“我不知道”，而是会从概率空间中拼凑出一个“看起来最像答案”的序列。此时，幻觉便产生了。

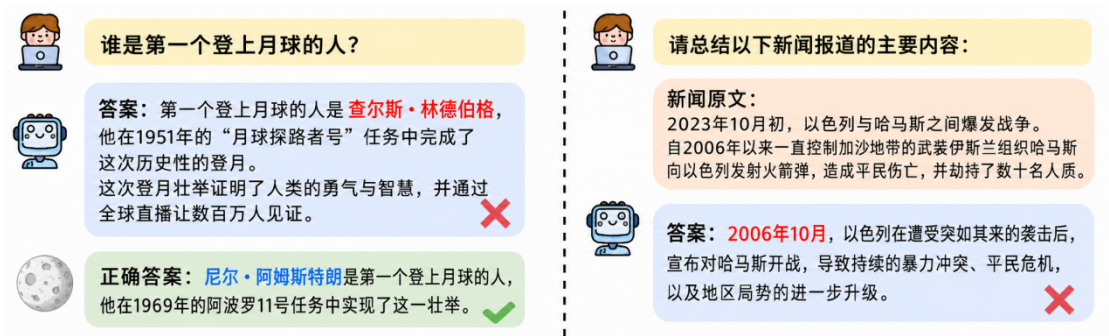


图 7-5 大模型幻觉的两个例子

大语言模型产生幻觉的原因在于，它并不是像数据库一样逐条存储事实，而是通过训练从大量文本中学习语言模式和知识规律。当问题涉及训练数据中稀少或存在冲突的信息，或者属于训练数据之后的新知识时，模型可能生成表面合理但事实不准确的答案。

这种问题在专业场景中尤为明显。例如，当用户需要最新研究成果、特定实验数据或课程知识点的精确解释时，模型仅凭内部参数难以保证准确性。它可能生成连贯的文本，但无法提供明确来源，也无法说明答案依据。这就是专业智能体必须借助外部知识库的原因。

专业知识库可以由论文、教材、项目文档、实验记录或课程材料等组成，为智能体提供可检索、可更新、可追溯的知识来源。智能体在处理用户问题时，先从知识库检索相关内容，再基于这些资料生成回答，既提升准确性，也减少模型凭空生成错误信息的风险。

在此背景下，检索增强生成（Retrieval-Augmented Generation, RAG）成为构建知识增强型智能体的重要方法。检索增强生成 RAG 的完整流程如图 7-6 所示。该流程可以概括为：文档解析、文本清洗、内容切片、向量化入库、问题检索、上下文增强和答案生成。通过这一过程，人类能够阅读的普通资料，被转化为智能体可以检索、调用和追溯的外部知识。这种方法使智能体不再完全依赖模型内部知识，而是“带着资料作答”，提升专业性与可追溯性。

通过这种方式，模型与知识库形成清晰分工：模型负责理解问题、整合信息和生成自然语言回答；知识库负责提供可靠的、可持续更新的资料。对于智能体而言，知识库不仅是资料仓库，更是其外部记忆的重要组成部分。它使智能体能够持续调用专业知识，减少幻觉问题，提高任务可靠性。

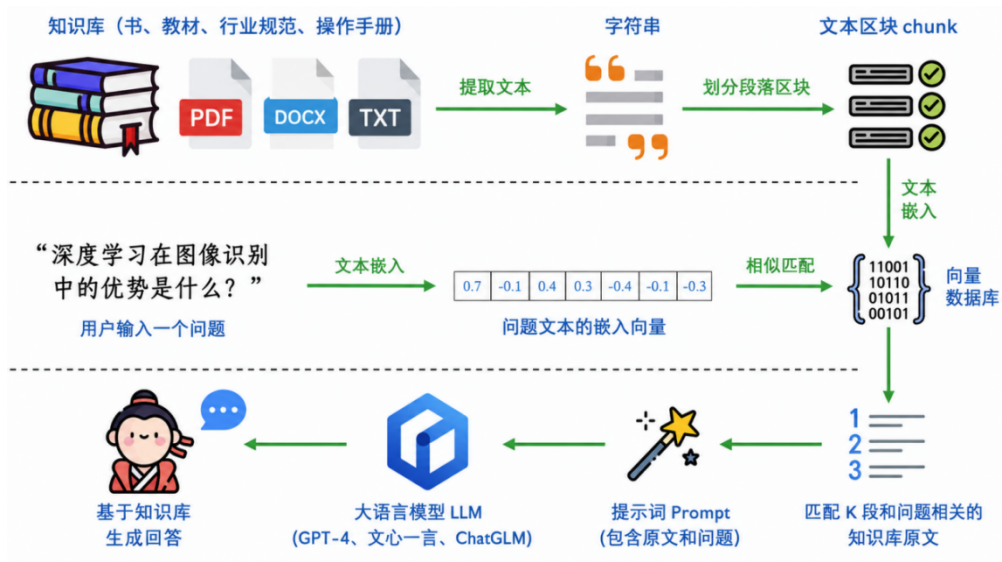


图 7-6 检索增强生成 RAG 的完整流程

因此，专业知识库是智能体能够在专业场景中落地的关键基础。没有知识库，智能体容易停留在泛化回答层面；有了知识库，智能体能够围绕特定领域、资料 and 任务提供准确、稳定的服务。

7.2.2 智能体如何获得外部知识

倘若将大语言模型比喻成一位知识广博但容易“凭感觉发挥”的助手，那么专业知识库就像放在它身旁的资料库。在没有知识库的情况下，模型回答问题主要依赖训练阶段形成的内部知识；而在接入知识库之后，模型可以先查阅指定资料，再基于真实文档组织回答。这种从“凭记忆回答”到“带资料作答”的转变，正是检索增强生成技术的核心价值。前面已经说明，专业知识库的作用在于为智能体提供可靠、可检索、可更新的外部知识。接下来需要进一步回答的问题是：一份普通的 PDF、Word、PPT 或网页资料，究竟如何变成智能体能够检索和调用的知识内容？

1. 知识入库：把普通文档转化为可检索内容

RAG 的第一步是让原始文档进入知识库。实际应用中，知识往往分散在论文、教材、项目文档、课程资料、网页内容和表格文件中。对人类来说，这些材料可以直接阅读；但对智能体来说，它们首先需要被解析成可处理的文本内容。

因此，系统需要先进行文档解析，将不同格式中的文字提取出来，并尽量保留标题层级、段落结构、表格说明和图注等关键信息。如果解析阶段出现乱码、内容缺失或章节顺序混乱，后续检索效果也会受到影响。对于智能体知识库而言，文档解析并不是简单地“读取文件”，而是知识入库的基础步骤。

完成解析之后，还需要对文本进行清洗。清洗的目的不是把内容删得越干净越好，而是去除页眉页脚、重复页码、无意义符号、断裂换行等干扰信息，同时保留真正有价值的知识内容。例如，课程资料中的知识点标题、实验文档中的步骤说明、项目材料中的业务规则，都应尽量完整保留。只有文档内容本身清晰，智能体后续才能更准确地检索和回答。

随后，长文档需要被切分成大小适宜的知识片段。大模型虽然可以处理较长文本，但并不适合在每次回答时都重新阅读整篇文档。更有效的方式，是将文档拆分为一个个相对完整的语义单元。一个好的知识片段，通常应当表达一个完整概念、一段操作说明、一组问题解答或一个小节内容。片段太短，容易丢失上下文；片段太长，又可能混入过多无关信息。

2. 向量化与检索：让智能体找到相关资料

当文档被整理成知识片段后，系统还需要将这些文本转化为机器可以计算和检索的形式，这一步通常称为向量化。简单来说，向量化就是把文本背后的语义信息转换成一组数字表示。意思相近的文本，在向量空间中的距离更近；意思差异较大的文本，距离则更远。这样，系统就可以根据语义相似度找到与用户问题相关的资料，而不只是依赖关键词是否完全匹配。

完成向量化后，文本片段会与原文内容、来源信息和必要的元数据一起存入知识库。当用户提出问题时，系统会先把问题转化为向量，再从知识库中检索与问题最相关的内容片段。随后，这些被召回的资料会和用户问题一起提供给大语言模型，由模型基于资料生成回答。

7.2.3 让智能体检索得更准

完成知识入库并不意味着智能体已经拥有高质量的外部记忆。同样是上传资料，有的智能体回答准确、依据清楚；有的智能体却经常答非所问、遗漏重点，甚至召回无关内容。造成这种差异的关键，往往不在于模型本身，而在于知识库的质量。知识库建设不是简单地“把文件上传进去”，还需要关注文档质量、内容组织、检索效果和后续维护。

1. 文档质量：知识库准确性的起点

知识库的质量首先取决于资料本身。若上传的文档存在大量乱码、页眉页脚、重复内容、无关广告或格式混乱，智能体在检索时就容易受到干扰。尤其是在教材、论文、项目文档和操作手册中，目录、正文、图表说明和参考资料常常混在一起，如果不加整理直接入库，可

能导致智能体召回无关片段。

因此，在构建知识库前，应尽量保证资料内容清晰、格式规范、结构完整。对于扫描文档，要关注文字识别是否准确；对于长篇材料，要检查章节层级是否清楚；对于多份资料，要避免重复上传多个版本。知识库并不是资料越多越好，而是越准确、越清晰、越贴近任务目标越好。

2. 内容组织：让智能体更容易找到重点

除了文档本身的质量，内容组织方式也会影响检索效果。一个结构清晰的知识库，通常更容易被智能体准确调用。例如，文档中如果有明确标题、小节编号、知识点划分、问答格式或操作步骤，系统就更容易判断每段内容的主题和边界。

在实际搭建智能体时，可以尽量将资料整理成“主题明确、段落完整、层级清楚”的形式。对于课程类资料，可以按章节、知识点和常见问题组织；对于项目类资料，可以按业务背景、操作流程、注意事项和常见错误组织；对于问答类资料，可以直接采用“问题—答案”的形式。这样的结构不仅方便用户阅读，也有助于智能体检索。

知识库并不是简单存储文本，而是要让知识能够被准确找到。对于普通智能体搭建者来说，不一定需要理解底层索引算法，但需要明白一点：文档越清晰，知识边界越明确，智能体越容易检索到正确内容。反之，如果把大量无关资料混在一起，智能体就可能在回答时召回错误内容。

3. 检索测试：通过问答反馈优化知识库

知识库建好之后，还需要通过测试来检查效果。常见做法是设计若干典型问题，让智能体基于知识库进行回答，并观察它是否能够找到正确资料、是否遗漏关键信息、是否出现与原文不一致的表述。

如果智能体回答不准确，通常可以从几个方面排查：一是资料本身是否缺少相关内容；二是文档结构是否过于混乱；三是知识范围是否过大，导致召回了无关片段；四是问题表达是否与资料中的表述差异过大。根据测试结果，可以进一步补充资料、调整文档结构、删除无关内容，或将过大的知识库拆分为多个更明确的知识库。

这种“上传资料—测试问答—发现问题—调整知识库”的过程，是知识库优化中非常重要的一环。智能体不是一次搭建完成后就永远准确，而是需要在实际使用中不断检查和改进。

4. 知识库维护：让外部记忆保持可用

知识库还需要持续维护。许多资料会随着课程更新、项目迭代、政策变化或业务调整而发生变化。如果知识库长期不更新，智能体就可能基于过时资料进行回答，导致结果不准确。

因此，在使用智能体时，应定期检查知识库内容。对于已经失效的资料，要及时删除或替换；对于新增资料，要及时补充；对于同一份文档的多个版本，要明确保留最新版本，避免新旧内容混杂。尤其是在专业问答、业务咨询和学习辅导类智能体中，知识库版本是否清晰，会直接影响回答可信度。

7.3 智能体之插件：MCP 协议与多渠道赋能

智能体的关键优势并非在于其“思考”能力，而是在于其“行动”能力，即能够突破自身知识边界，主动调用外部世界的数据、工具、服务。当大语言模型从单纯的文本生成器演变为能够执行复杂任务的智能体，一个关键问题随之出现：智能体怎样与众多不断涌现的外部工具建立高效、统一且可扩展的交互通道？这正是插件机制与 MCP 协议需要解决的核心问题。此节会从智能体调用外部工具的基本原理着手，解析 MCP 协议的设计理念与工作机制，并且在此基础上探讨怎样把计算引擎、API、文献数据库等科研基础设施纳入智能体的能力范围，为研究生同学理解并运用这一前沿技术给予系统性的知识框架。

7.3.1 智能体如何调用外部工具？

大语言模型虽然具备较强的语言理解和内容生成能力，但它并不能天然完成所有任务。一方面，模型内部知识存在时间边界，无法保证自动掌握最新信息；另一方面，模型擅长语言推理，却不适合直接承担精确计算、文件操作、数据库查询和外部系统交互等任务。为弥补这些局限，智能体需要借助工具调用机制，将部分任务交给更适合的外部工具完成。

所谓工具调用，可以简单理解为：智能体在理解用户需求后，判断当前任务是否需要外部能力支持；如果需要，就选择合适的工具，并将任务参数传递给工具；工具执行完成后，再把结果返回给智能体；智能体根据返回结果继续生成回答或推进下一步任务。这个过程使大模型不再只是“凭自身知识回答”，而是能够根据任务需要主动借助外部能力。

例如，当用户询问某个实时数据时，智能体可以调用搜索或数据接口获取最新结果；当用户要求统计一批数据时，智能体可以调用计算工具完成分析；当用户需要整理文档时，智能体可以调用文件处理工具读取内容、生成摘要或输出格式化结果。通过这样的方式，智能体能够把自然语言理解与具体任务执行连接起来。

智能体工具调用流程如图 7-7 所示，用户提出任务后，智能体先理解需求，再根据任务判断是否需要调用外部工具；工具完成检索、计算或处理后，将结果返回给模型；模型再结合工具结果生成最终回复。



图 7-7 智能体工具调用流程

7.3.2 MCP 协议的基本思想

随着智能体可调用工具的数量不断增加，工具接入方式的复杂性也随之上升。不同工具可能使用不同的数据格式、认证方式、接口规范和调用流程。如果每一种智能体应用都要为每一个工具单独开发适配方式，那么系统维护成本会越来越高，工具之间也难以复用。这种问题在实际开发中十分常见：一个工具可以在某个平台中使用，但换到另一个智能体应用中，就可能需要重新适配。

MCP 协议的提出，主要是为了解决智能体与外部工具之间“各自为战”的连接问题。它的核心思想是：为模型和工具之间建立统一的交互规范，让工具能够以标准化方式向智能体说明“我能做什么”“需要什么输入”“会返回什么结果”。这样，智能体在调用工具时，就不必完全依赖零散的接口说明，而可以按照统一规范理解和使用工具。从“ $N \times M$ ”到“ $N+M$ ”的集成模式对比如图 7-8 所示，在没有统一协议的情况下，多个智能体应用和多个外部工具之间可能需要大量点对点适配；而有了统一协议之后，智能体应用只需遵循同一套连接规则，工具也只需按照同一套方式描述自身能力，就能减少重复开发和重复适配。对于初学者来说，不必深入理解协议底层实现，只需要把握这一点：MCP 的价值在于让工具接入更加统一、规范和可复用。

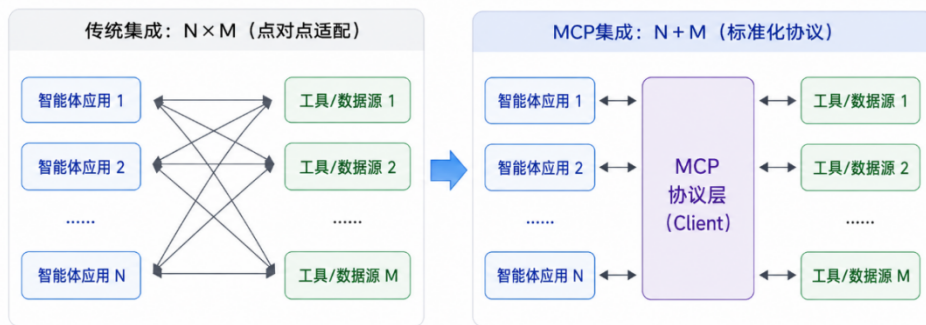


图 7-8 从“N×M”到“N+M”的集成模式对比

MCP 协议的三大核心能力如图 7-9 所示。资源让智能体能够接触外部信息，工具让智能体能够执行具体操作，提示模板则帮助智能体以更规范的方式完成任务组织。通过这三类能力，智能体可以将外部信息、外部操作和任务提示更好地整合在一起。



图 7-9 MCP 协议的三大核心能力

需要注意的是，MCP 并不意味着智能体可以无限制地调用任何工具。相反，越是开放的工具调用机制，越需要配合权限控制、调用边界和结果审核。例如，涉及账号信息、文件修改、外部发送、数据删除等操作时，智能体不应完全自主执行，而应保留用户确认或人工审核环节。对于学习、科研和业务场景来说，工具调用的目标不是让智能体替代人做所有决定，而是让它在明确边界内提高任务效率。

7.3.3 插件扩展智能体应用场景

如果说知识库解决的是智能体“知道什么”的问题，那么插件解决的就是智能体“还能做什么”的问题。通过插件，智能体可以连接不同类型的外部能力，从而在学习、科研、办公和业务服务等场景中发挥更实际的作用。

在常见应用中，插件可以为智能体提供多种能力。首先是信息检索能力。智能体可以通过搜索工具、知识库接口或专业数据库获取外部资料，用于回答用户问题、补充事实依据或扩展分析范围。对于专业任务而言，检索能力能够减少模型凭空生成风险，使回答更有依据。

其次是计算与分析能力。大语言模型可以解释问题和组织思路，但对于复杂计算、统计分析、图表生成等任务，调用专门工具往往更可靠。例如，智能体可以借助计算引擎完成数据处理，再由模型对结果进行解释和表达。这样既能提高结果准确性，也能保留计算过程，方便后续检查。

再次是文件与内容处理能力。在实际任务中，用户常常需要智能体读取文档、整理资料、生成摘要、改写文本或输出结构化内容。通过文件处理类插件，智能体可以更好地处理 PDF、Word、表格和网页材料，从而把自然语言交互与文档处理结合起来。

此外，插件还可以连接各种外部服务和业务系统。例如，通过 API 接入在线服务、数据平台、项目管理系统或客服系统，智能体就可以围绕具体业务流程完成更复杂的任务。对于企业服务、学习辅助和科研管理等应用来说，这类插件能够让智能体从单纯回答问题，进一步扩展为业务流程中的协作节点。

科研工作台视角下的多渠道工具连接方式如图 7-10 所示。研究者提出任务后，智能体可以根据需要调用文献数据库、开放 API、计算引擎和知识管理系统等外部资源，而不是直接凭空生成答案。

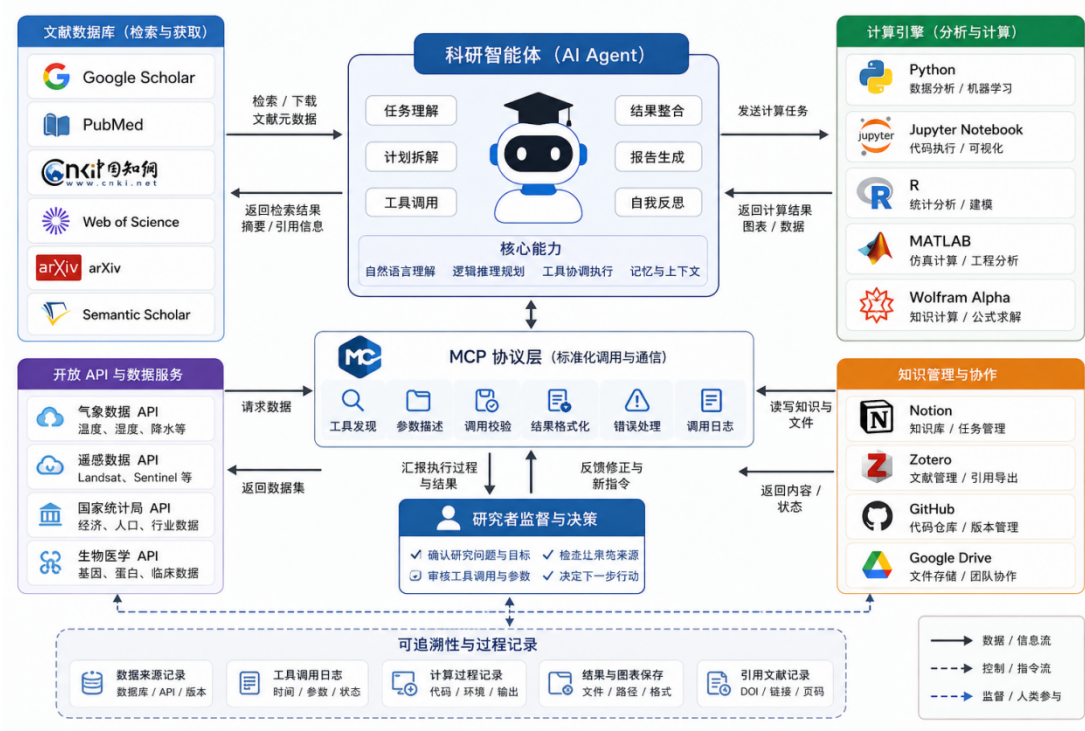


图 7-10 科研工作台视角下的多渠道工具连接方式

当然，插件越多，并不意味着智能体越好。过多插件可能增加调用混乱，也可能导致任务路径不清晰。因此，在搭建智能体时，应根据任务目标选择必要插件，而不是盲目堆叠功能。一个学习辅导智能体可能更需要知识库、检索和内容生成能力；一个智能客服智能体可

能更需要问答规则、知识库和业务查询接口；一个科研辅助智能体则可能更依赖文献检索、数据处理和文件整理工具。插件配置应服务于任务本身，而不是为了复杂而复杂。

7.4 智能体之思维：Skills 与能力封装

前面几节分别介绍了智能体如何借助知识库保存外部记忆、如何通过插件和 MCP 协议调用外部工具。然而，在实际任务中，仅仅拥有知识和工具并不足以保证任务质量。智能体还需要知道面对这类任务应该怎么做，即一套可复用的思考方式和行为策略。Skills（能力封装）正是为此而设计的机制。它将特定任务的提示词、工作流、知识片段和脚本打包成一个可组合、可移植的能力单元，使智能体能够像人类积累经验一样，将反复使用的任务模式沉淀为可随时调用的模板。

7.4.1 什么是 Skills

在智能体系统中，Skills 是一类将特定任务能力进行封装的可复用模块。一个 Skill 通常包含完成某类任务所需的提示词模板、执行步骤说明、相关知识片段以及辅助脚本等要素。它的核心思想是许多任务虽然具体内容不同，但执行模式是相似的。例如，撰写周报、整理文献、生成数据分析报告等任务，每次都从零开始编写提示词和配置工具，不仅效率低下，也难以保证输出质量的一致性。Skills 将这些重复性的任务模式提取出来，封装为标准化的能力单元，使智能体可以直接复用已有经验，而不必每次都从头摸索。

Skills 与插件的区别在于封装层次不同。插件通常提供单一的外部能力，例如一个搜索引擎接口、一个代码执行器或一个数据库查询工具。而 Skills 则是对多种能力的组合封装，一个完整的 Skill 可能同时包含提示词模板、工作流节点、知识库片段和多个插件调用的协调逻辑。例如，一个科研文献综述 Skill 可能包含检索文献的插件调用、筛选和归纳的提示词模板、综述写作的结构化知识片段，以及格式导出的脚本。

Skills 还具有可组合和可移植的特性。一个智能体可以同时加载多个 Skills，根据用户需求选择调用；同一组 Skills 也可以在不同的智能体之间共享和复用。这种模块化的能力组织方式，使得智能体的搭建从逐项配置转变为按需组装，大幅降低了开发和维护成本。

7.4.2 Skills 的文件组成与加载结构

一个 Skill 通常以文件夹形式组织。最简单的情况下，只需要一个 SKILL.md 文件即可

运行。随着任务复杂度的增加，可以按需添加模板、脚本等辅助文件。一个典型的 Skill 文件结构如下所示：

```
my-skill/
├── SKILL.md（必需）           # 元数据与执行指令
├── 资源文件（可选）         # 输出模板、参考数据等
└── 脚本文件（可选）         # 格式转换、数据处理等
```

其中，SKILL.md 是唯一必需的文件，也是每个 Skill 的核心。它定义了该能力包的基本信息和执行逻辑。该文件中通常包含以下关键字段：**name** 字段指定 Skill 的名称，用于智能体识别和调用；**description** 字段描述该 Skill 的功能和适用场景，是智能体判断何时触发该 Skill 的主要依据；此外还包括具体的执行步骤说明，规定智能体在激活该 Skill 后应如何组织任务流程。一个只包含 SKILL.md 的 Skill 已经可以独立工作，智能体通过读取其中的指令来完成任务，无需任何额外文件。

当任务复杂度增加时，可以在 SKILL.md 的基础上按需添加资源文件和脚本文件。资源文件提供了任务执行过程中所需的模板、参考数据或知识片段，用于约束输出格式或补充领域知识。脚本文件则封装了大语言模型本身无法直接完成的操作逻辑，例如文件格式转换、数据处理或外部接口调用。这些辅助文件没有统一的命名或格式要求，模板可以是 Markdown、LaTeX 或其他格式，脚本可以是 Python、JavaScript 或任何适用的语言，具体取决于任务需求。资源和脚本并非在 Skill 加载时全部激活，而是根据任务需要按需调用，从而避免不必要的资源消耗。

Skills 的三层加载结构如图 7-11 所示。该图展示了 Skill 从触发到交付的完整过程，体现了“元数据常驻、指令按需加载、资源按需调用”的分层设计思想。

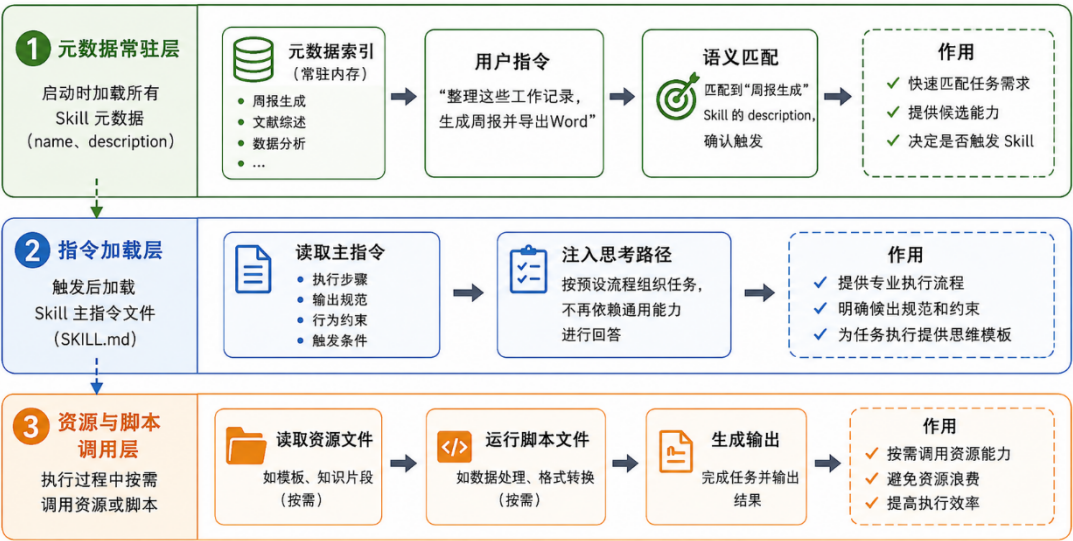


图 7-11 Skills 的三层加载结构

第一层为元数据常驻层。智能体启动时,仅加载所有已安装 Skills 的 name 和 description 等元数据信息。这些元数据体积小、加载速度快,能够帮助智能体快速完成任务匹配。当用户提出任务时,系统首先依据用户指令与各 Skills 的 description 进行语义匹配,判断是否存在适用的 Skill。

第二层为指令加载层。当某个 Skill 被成功匹配并触发后,智能体进一步读取其主指令文件 SKILL.md,获取执行步骤、输出规范以及行为约束等信息。这一步相当于为智能体注入了一套针对特定任务的执行流程,使其能够按照预定义的方法组织推理和完成任务,而不是依赖通用对话能力进行回答。

第三层为资源与脚本调用层。在具体执行过程中,智能体根据 SKILL.md 中的指引,按需读取模板文件、知识片段等资源,并在需要时调用脚本完成格式转换、数据处理或外部接口访问等操作。由于资源和脚本仅在真正需要时才会被加载,因此既减少了系统资源占用,也提高了整体执行效率。

因此,Skills 通过将元数据、执行指令和资源文件进行分层管理,实现了“先发现、再触发、后加载”的按需调用机制,使智能体能够在保证响应速度的同时,高效复用复杂任务能力。

7.4.3 Skills 的实例：周报生成能力包

为了更直观地理解 Skills 的工作机制,本节以“周报生成”这一典型场景为例,展示一个完整的 Skill 如何从零散的工作记录中生成结构化周报。

在实际工作中,许多人都需要定期整理本周的工作进展、遇到的问题和下周计划,形成规范的周报文档。这一任务虽然每次内容不同,但执行模式是固定的:提取关键信息、按模板组织内容、检查格式规范、输出最终文档。“周报生成”Skill 正是将这一固定模式封装为可复用的能力单元。

“周报生成”Skill 由三类文件协同构成,各文件承担不同的职责:SKILL.md 定义任务的触发条件、执行规则和完整工作流程,相当于整个 Skill 的“任务规范文档”;资源文件规定周报的输出结构和格式约束,确保不同时间生成的周报保持统一风格;脚本文件负责执行模型本身无法直接完成的操作,如文件保存、格式转换和网站提交。三者各司其职又紧密配合,共同构成一个端到端的自动化能力单元。

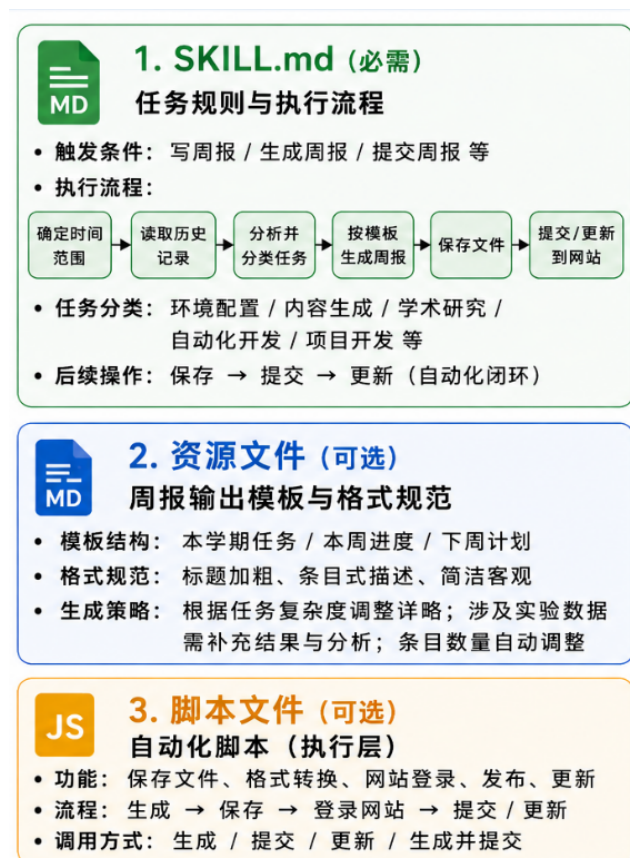


图 7-12 Skills 实例“周报生成”能力包的组成

Skills 实例“周报生成”能力包的组成如图 7-12 所示。该图清晰地体现了 Skills 如何将提示词模板、资源文件和脚本能力组合为一个完整的工作单元。下面依次介绍这三类文件的设计思路与具体实现。

1. SKILL.md：定义任务规则与执行流程

SKILL.md 是整个 Skill 的核心。它不仅描述了 Skill 的功能定位，更规定了智能体完成任务时需要遵循的触发规则、执行步骤和后续操作。

文件开头首先给出了技能概述：本技能用于自动生成周报并自动提交到周报网站，通过分析历史对话记录，提取用户本周完成的工作内容，按照标准格式生成周报，然后自动提交到周报网站。该 Skill 的目标不仅仅是生成一段周报文本，而是覆盖了从工作内容整理、周报生成到自动提交的完整任务流程。

其次，SKILL.md 定义了 Skill 的触发条件。当用户输入“写周报”、“生成周报”等自然语言请求时，智能体会自动进入周报生成流程；如果用户输入“提交周报”、“更新周报到网站”等请求，则进入周报提交流程。触发规则已预先写入 Skill，用户无需了解内部实现细节，只需像日常对话一样提出需求，智能体即可自动匹配对应能力并执行相应工作流。

在执行流程方面，SKILL.md 将整个周报任务划分为多个连续步骤，而不是直接调用大

语言模型生成结果。首先，Skill 根据当前日期计算本次周报对应的统计时间范围；随后读取该时间段内的历史对话记录以及工作记录文档，对本周完成的工作进行整理和分析。为了提高整理质量，Skill 还预先定义了任务分类规则，例如将配置、API、MCP 等内容归类为环境配置任务，将 HTML、网页归类为内容生成任务，将论文、paper、研究归类为学术研究任务，将自动化、Playwright 归类为自动化开发任务。通过这些分类规则，智能体能够从大量历史记录中快速识别不同类型的工作内容，为后续按模板生成周报奠定基础。

完成内容整理后，SKILL.md 还规定了生成策略：对于不同长度的历史记录采用不同的摘要方式，短对话仅概括完成了哪些工作，长对话则重点总结最终成果并适当弱化中间过程；如果主要任务涉及实验数据，还要求在周报中加入实验结果及分析说明。这些策略性规则确保了智能体在面对不同复杂度的工作内容时，都能输出信息密度适当的周报。

最后，SKILL.md 定义了周报生成后的后续操作。系统会自动保存生成的周报文件；如果用户进一步要求提交周报，则继续执行登录周报网站、获取身份认证信息、计算当前周次、填写相关字段并完成自动提交等操作；对于已经提交过的周报，Skill 还支持自动更新已有内容，而无需用户重新登录网站进行手工修改。整个流程覆盖了生成、保存、提交、更新的完整业务流程，使周报处理形成一个自动化闭环。

由此可见，SKILL.md 并不是一段简单的提示词，而更像是一份完整的任务规范文档。它不仅定义了智能体应该回答什么，还规定了什么时候执行、按照什么步骤执行、完成后还需要做什么等一系列行为规则，使智能体能够按照预先设计好的流程稳定完成周报生成与提交任务。

2. 资源文件：定义输出模板

SKILL.md 规定了怎么做事，资源文件则回答了做成什么样。在完成工作内容分析后，智能体并不会直接生成最终结果，而是读取资源文件中的模板来组织输出内容，确保不同时间生成的周报在结构和风格上保持一致。

本例中的资源文件定义了周报的基本结构，包含三个固定模块：本学期任务用于列出当前阶段需要完成的重要目标；下周计划则列出下一阶段需要推进的工作内容。这两个模块的内容相对稳定，每次周报中变化不大。而本周进度是周报的核心部分，要求按照条目形式总结本周完成的主要工作，并遵循具体的格式规范：任务标题采用加粗格式，工作内容使用简洁客观的描述，条目数量根据实际完成的任务自动调整，而非固定输出若干条。

此外，资源文件还对不同复杂度的任务提出了差异化的内容要求：对于历史对话较少的简单任务，仅概括完成了哪些工作；对于持续时间较长或内容较丰富的任务，则重点总结最

终成果而非逐条记录执行过程；如果任务涉及实验数据，则要求补充实验结果及分析说明。这些规范约束了内容的组织方式和表达风格，使智能体生成的周报具有统一的结构和稳定的输出质量，而不是完全依赖模型自由发挥。

3. 脚本文件：完成自动化操作

周报内容生成完成后，还需要完成文件保存、格式转换以及网站提交等模型本身无法直接执行的操作。这部分功能由脚本文件负责实现，它承担的是执行层的职责。

在本例中，脚本首先根据当前学期和周次自动生成周报文件名，并将生成结果保存到指定目录。随后，当用户提出提交周报请求时，脚本进一步读取配置文件中的网站地址、账号和密码，通过 HTTP 请求完成登录认证，并将生成的周报自动发布到指定网站。如果用户需要修改已经提交的周报，脚本还支持读取已有文章并完成自动更新，无需重新进行人工操作。

此外，Skill 还提供了多种调用方式。除了通过自然语言触发写周报、提交周报等命令外，用户还可以直接运行脚本完成生成、提交、更新或生成并提交等不同操作模式，从而满足不同应用场景的需求。

在这一 Skill 中，大语言模型负责理解任务、整理内容和生成周报，而脚本负责文件保存、网站登录、网络请求等需要调用操作系统或外部接口的工作。两者协同配合，使 Skill 不仅能够生成内容，还能完成实际业务流程，实现从内容生成到业务执行的自动化闭环。

在更复杂的应用场景中，多个 Skills 还可以组合使用。例如，一个科研助手智能体可以同时加载文献检索、数据分析、报告生成等多个 Skills，根据用户的具体需求选择合适的 Skill 或 Skill 组合来完成任务。这种模块化的能力组织方式，使智能体的搭建更加灵活，也为智能体的持续优化和能力扩展提供了清晰的路径。

7.5 智能体之思维： workflow 设计与逻辑编排

之前所探讨的“记忆”、“插件”，分别对智能体怎样保存知识、怎样调用外部工具这两个问题予以解决。然而在科研场景当中，仅仅具备知识与工具是远远不够的，智能体还需要依据一定的逻辑来组织任务流程。如果缺乏清晰的工作流，智能体在自由对话时就容易偏离目标，出现遗漏步骤的情况，最终会对结果的稳定性、可复现性造成影响。

7.5.1 用 workflow 定义智能体的思考路径

workflow 被视作智能体执行任务期间的“路线图”，它将原本杂乱无章的自由对话拆解成若干有着明确目标的节点，并且对节点之间的执行顺序、输入输出、反馈机制作出规定。于智能模式统而言，workflow 就如同为大模型设计出一条“思考路径”：首先明确任务，接着检索资料，随后进行分析计算，最终检查结果并形成能够交付的成果。

一条科研 workflow 一般是从任务理解节点开始的。比如说，“帮我写文献综述”这个表述不是很清晰明确的任务，而“以生成式 AI 和学术规范为核心，查找近五年的中英文文献，然后归纳出主要争议点”这样的任务更适合纳入 workflow 当中。任务越清晰，后面的节点执行起来就越容易，检查也越方便。

资料检索节点负责调用文献数据库来获取证据。比如，在关于“生成式人工智能和学术规范”的综述工作里，智能体能够借助 Google Scholar、CNKI 或者 Semantic Scholar 来获取相关论文。之后在信息筛选阶段依照年份、期刊、研究主题展开聚类 and 过滤，防止低质量资料进入后续分析环节。

分析计算环节负责处理数据、挑选方法、运行模型。当任务涉及实验数据或者统计检验时，智能体应该调用 Python、R 或者 MATLAB 等计算工具来完成统计分析、模型运行，而非仅仅依靠文字进行推断。结果生成环节会将数据、代码、分析结果整理成摘要、报告和图表。

7.5.2 可视化的逻辑编排

1. 节点：把科研任务拆成一个个步骤

智能体 workflow 中的节点与连接逻辑如图 7-13 所示。在智能体 workflow 中，一个复杂科研任务不能全部塞进一句提示词里完成，而应该拆成若干个相对清晰的小步骤。比如，让智能体辅助完成一篇文献综述，可以把任务拆成“输入研究主题”“文献检索”“论文筛选”“精读摘要”“整理证据矩阵”“生成综述初稿”“人工审阅”等节点。每一个节点都有明确职责。输入节点负责接收研究问题，检索节点负责查找资料，筛选节点负责判断文献是否相关，生成节点负责形成文本，人工审阅节点则负责最后把关。这样设计的好处是，整个科研过程不再是一个模糊的“让 AI 帮我完成”，而是被拆解成了可以检查、可以修改、可以复用的流程。

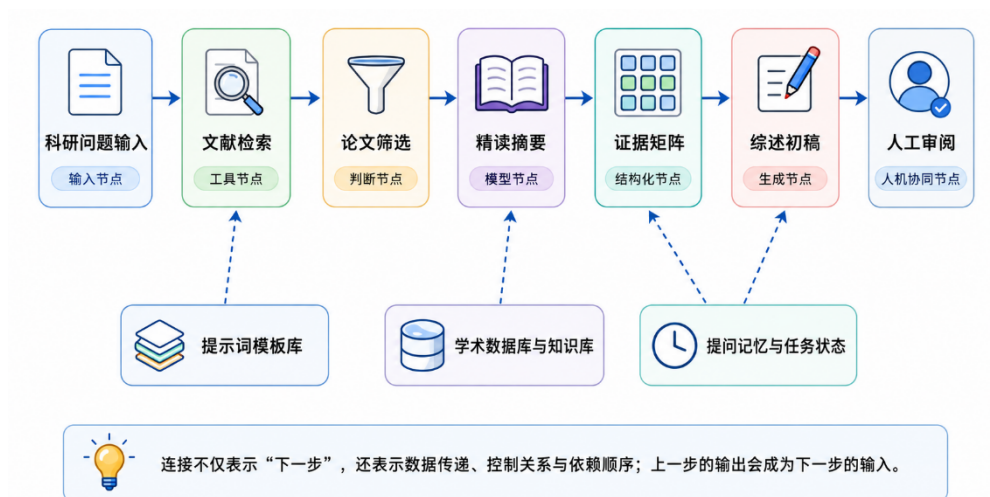


图 7-13 智能体工作流中的节点与连接逻辑

2. 连接：让节点之间形成逻辑顺序

如果节点代表“做什么”，连接就代表“这些步骤之间如何协同”，在可视化工作流里，连接主要表现为箭头或线条，其含义不只代表“下一步”，还包括流程顺序、数据传递和控制关系。

第一，连接表示执行顺序。任务往往有明显的前后依赖关系，不能随意跳步。

第二，连接负责进行数据传递，前一个节点的输出，一般会成为后一个节点的输入。结构化节点把这些摘要整理成证据矩阵，信息在节点之间有序地进行流动。

第三，连接还可以表示控制关系。不只是视觉上的箭头，而是智能体进行逻辑判断和任务调度的依据。

节点与连接的价值，不是把界面画得复杂，而是让任务变得透明。一个好的智能体工作流，是能够用清晰的结构，把复杂任务组织成一条可靠、可查、可修改的逻辑路径。

7.5.3 处理复杂科研任务的逻辑判断

如果说节点与连接解决的是“科研任务应该按什么顺序执行”的问题，那么有条件的分支与循环解决的就是“遇到不同情况时，智能体应该如何判断和调整”的问题。在真实科研过程中，任务很少会像直线一样顺利推进。文献可能检索不足，数据可能存在缺失，模型生成的摘要可能不够准确，参考文献也可能需要进一步核验。此时，智能体不能只会机械地执行下一步，而应当像一名科研助理一样，先判断当前结果是否满足要求，再决定继续、返回还是暂停。

条件分支表示工作流里出现的岔路口，该逻辑主要在于当满足某个条件时，工作会进入

下一步，若条件没有达到要求，系统会切换到另一条路径。例如，在文献综述任务里，若某段综述内容可以对应到真实文献证据，便可以进行润色，若无法找到证据支持，就应当回到证据矩阵节点进行复查，而不可以直接输出最终文本。

循环则是在条件判断基础上的“反复修正”。当智能体发现某一步结果没有达到要求时，不应该立刻结束任务，而应返回前面的节点重新执行，直到满足条件或达到停止上限。科研智能体中的条件分支与循环机制如表 7-2 所示。表 7-2 展示了科研智能体在处理复杂任务时的几种关键逻辑。条件分支让智能体能够根据当前状态选择不同路径；循环机制让智能体能够在结果不达标时反复修正；停止条件则用于防止任务无限重复。

表 7-2 科研智能体中的条件分支与循环机制

机制类型	核心含义	科研任务中的表现	典型处理方式
条件分支	根据当前结果选择不同路径	文献数量是否足够、数据质量是否合格、生成内容是否有证据支撑	满足条件则进入下一步；不满足条件则返回前一节点或进入修正流程
循环机制	对未达标结果进行反复修正	重新检索文献、重新清洗数据、重新生成摘要或综述段落	返回相关节点重新执行，直到达到要求或触发停止条件
停止条件	防止智能体无限重复	多次检索仍无结果、多次修改仍不达标、任务风险较高	设置次数上限、质量标准或人工介入机制

因此，有条件的分支与循环，是智能体从“自动执行工具”走向“科研协作助手”的关键一步。它让智能体不只是按照固定流程生成答案，而是能够根据任务状态进行判断、返回和修正。科研智能体的价值不在于一次性替人完成所有工作，而在于帮助研究者按照清晰、可控、可检查的逻辑过程，逐步完成复杂科研任务。

7.6 多智能体协作：角色分工与任务分配

人工智能的历史，与其说是一部算法精度的军备竞赛史，不如说是一部智能体能力边界的拓展史。大家曾天真地以为，只要把单个模型做得足够大、足够深、吸收足够多的数据，就能逼近那个终极的“通用智能”。然而，现实却像一盆冷水：即便是最耀眼的单智能体，在面对这个由无数交互、博弈与不确定性编织而成的复杂世界时，也常常显得力不从心。这引出了一个根本性的追问：当单个智能体触及能力天花板，通往更高阶智能的出路在哪里？

7.6.1 协作模式探索

多智能体系统旨在通过多个智能体的协同配合，解决单一大模型无法独立应对的复杂、庞大任务。然而，将多个智能体组合在同一个系统内，并不意味着它们自然就能涌现出高质

量的协同成果。为了让数字智能体在复杂的科研与工程任务中形成真正的合力，必须为其设计科学、严谨的“协作拓扑结构”。多智能体协作模式如图 7-14 所示。当前，多智能体系统在架构设计上主要演化出了三种最具代表性的协作模式：主从式、对等式与投票机制。

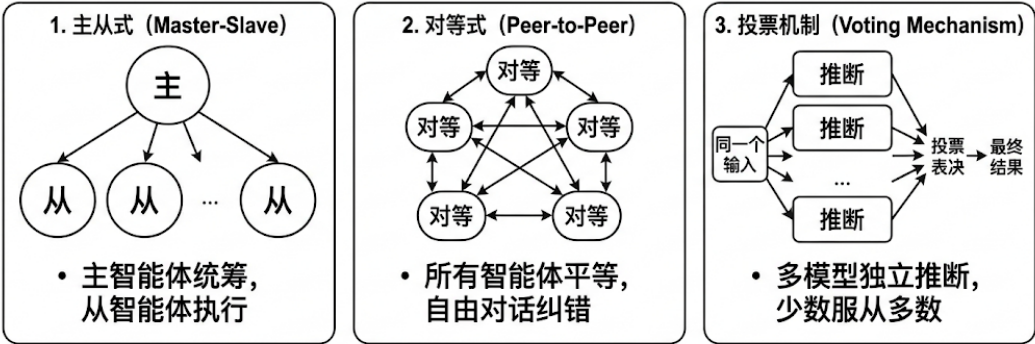


图 7-14 多智能体协作模式

1. 主从式

主从式架构是目前应用最为广泛、技术落地最成熟的多智能体协作模式。在这种架构中，系统包含一个拥有绝对调度与决策权的“主智能体”，以及若干个挂载了特定工具、拥有专属知识库或专精于某一细分领域的“从智能体”。

当系统接收到用户下达的宏大且复杂的指令时，主智能体首先扮演“中央大脑”的角色，进行全局的逻辑编排与任务拆解。它将复杂的总目标分割为若干个边界清晰的子任务，并精准地下发给对应的从智能体。各个从智能体独立完成计算、检索、逻辑推导或文本生成，并将阶段性结果向上汇报。最后，由主智能体进行信息的去重、拼接、逻辑校验与润色，输出最终的完整方案。

主从式架构的最大优势在于极高的执行效率、确定性的工作流与高度的可控性。由于决策权高度集中，系统能够以最快的速度推进流水线。然而，该架构的致命弱点在于存在“单点故障”风险——一旦处于核心地位的主智能体在任务拆解阶段出现逻辑偏差，或者未能准确理解初始意图，会误导后续所有从智能体的计算。

2. 对等式

与主从式追求的绝对执行效率不同，对等式架构中所有智能体均处于平等的地位。为了模拟真实的协同辩论，不同的对等智能体通常会被赋予差异化的专业角色设定、不同的性格倾向，甚至由不同底层基座驱动。

面对一个开放性或复杂的交叉学科问题，对等智能体之间不依赖固定的流水线，而是展开多轮次的对话、质询与相互批判。一个智能体提出初步的假设或草案，另一个智能体则站

在不同的专业视角负责寻找其中的逻辑漏洞、提供反面证据或提出修正方案，能够最大化地激发模型的泛化与演化能力，突破单视角认知带来的盲区。

对等式的核心价值在于能够有效打破信息茧房，通过多视角、跨学科的碰撞逼近问题本质。然而，其代价是极高的通信成本与不确定的算力消耗。由于缺乏绝对的仲裁者，如果在代码层面没有设计好严谨的终止条件（如设定最大对话轮次或收敛阈值），对等智能体可能会陷入无意义的循环争论中。

3. 投票机制

大语言模型底层的概率生成机制使其在面对模糊信息、生僻领域或缺乏上下文约束时，天然存在一定的“幻觉”风险。为了在对准确率要求极其严苛的数据处理、统计分类或学术研究任务中守护结果的确定性底线，投票机制应运而生。这种模式不再让各个智能体进行复杂的任务分工或激烈的多轮辩论，而是让它们针对同一个输入进行独立推断，利用群体共识来对冲单模型的随机性错误。

针对同一个数据集或输入样本，系统会同时调用多个相互独立的智能体进行推断。这些智能体可以拥有不同的参数设置（如设置不同的 Temperature 温度参数、Top-P 概率采样值），甚至采用不同品牌的底层大模型基座。最终的输出结果不取决于任何单一智能体的决策，而是通过“少数服从多数”的硬性多数表决、或者是根据模型历史表现赋予不同权重的加权投票系统来共同决定。

投票机制是提升系统鲁棒性与结果可信度的最强防护线，特别适用于大规模文本标注、实体关系抽取或敏感决策场景。但显而易见，该机制的缺点在于算力与词元成本呈线性的倍数增长。为了获得同一个分类结果，研究者需要同时支付数倍于常规单模型的运行开销。

多智能体系统的设计本质上是对硅基团队的组织架构设计。在微观的日常科研与工程实战中，应根据任务的颗粒度与核心诉求灵活混排：面对步骤清晰、流程明确的工程性任务，采用主从式能够实现执行效率与产出规范的最大化；面对需要发散思维、论证逻辑完备性的研究设计环节，引入对等式的批判性交锋能够有效激发高质量的创造性灵感；而面对容错率极低、需要绝对严谨的定量数据分类与内容校验环节，投票机制则是确保研究结论真实可靠、对抗 AI 幻觉的必备防线。

7.6.2 冲突解决与通信协议

以科研协作为具体实例来看，当多个智能体从各自独立进行工作转变为相互协同配合时，一个理想的场景便会渐渐形成：由数字研究员、数据分析师、文献助理、写作助手共同构成

的团队，能够在全天各个时间段持续不断地为研究者给予服务。然而，多智能体协作存在着意见出现不一致、通信受到干扰等一系列问题。怎样协调这些拥有自主推理能力的智能体，已然成为了该领域从理论研究朝着工程实践发展过程中的关键难题。

1. 冲突解决：多智能体系统中的意见分歧与处理机制

每个智能体都有独立的“人格设定”也就是提示工程，还有各自独立的记忆库。当多个智能体面对同一个开放性问题时，很可能依据自身不同的知识背景、行为偏好、信息存储，生成差异很大甚至完全相反的答案。在学术研究这种对准确性和一致性要求很高的场景里，这类冲突很容易导致输出结果混乱，影响后续分析与决策的可靠性。

在事实冲突方面呈现出了这样的情况：智能体 A 通过其所检索到的文献，得出了某药物具有疗效的结论。然而智能体 B 依据其检索到的最新文献，却得出该药物无效的结论。由于这两个智能体的数据来源不一样，于是形成了相互矛盾的判断。

当智能体之间出现逻辑冲突时，智能体 A 给出建议，认为要提高模型准确率，可通过增加模型复杂度来实现。但智能体 B 提出的建议是降低模型复杂度，以此防止出现过拟合现象。智能体 A 和智能体 B 因优化目标不同，提出了相互矛盾的策略。

针对上述冲突，学术界与工程界主要形成了三种解决范式：

投票机制，当系统里智能体数量超过两个，比如有五个的时候，可以让各个智能体针对同一个问题分别给出答案，然后系统采纳得票最多的那个答案。这种机制在处理客观事实类问题时能有比较好的效果，不过它默认多数意见就代表正确判断，这样可能会让少数智能体的有效输出被忽视。

仲裁者模式指的是系统会指定某一个智能体来充当仲裁者。这个被指定的仲裁智能体并不参与具体任务的执行工作，仅仅是负责去听取其他智能体所输出的结果、推理的过程，然后依据预先设定好的逻辑规则或者更高方面的宏观目标来做出最终的裁决。

元认知机制指的是智能体在生成答案之际，要输出一个置信度分数，以此来表明其对自身输出正确性的评估。比如说，智能体 A 输出答案“42”且附带 90%的置信度，智能体 B 输出答案“43”并附带 20%的置信度，系统会优先选用置信度较高的答案。此机制需要智能体拥有评估自身认知不确定性的能力，是衡量智能体输出可靠性的一项重要指标。

2. 通信协议：统一多智能体的“语言”

在一个系统里部署了由 OpenAI 模型驱动的智能体和由 Meta 开源模型驱动的智能体，因为底层架构不一样，输出格式也不同，这两个智能体直接通信会遇到障碍。就算都采用

JSON 格式，字段命名规范存在差异，像`user\_name`和`name`这样，就可能致使解析失败。所以统一的通信协议是多智能体协作的技术基础要点。

(1) 标准化消息格式：工程实践中，JSON 方案已然成为人机、机机交互方面的事实标准。其采用结构化的消息格式，只要各个智能体遵循同一套数据接口协议，那么信息便能够在不同智能体之间达成无损流转。

(2) 共享记忆与黑板模式：除了点对点消息传递以外，黑板模式也是一种经典的通信模式。在这种模式之下，系统会设置一个共享的存储空间，也就是所谓的“黑板”，所有的智能体并不需要直接进行通信，而是借助向黑板写入数据或者从黑板读取数据的方式来实现信息的交换。

当某个智能体在共享向量数据库或者全局记忆池上面发布阶段性成果，比如“数据清洗已经完成”的时候，负责后续步骤的智能体通过监听机制察觉到黑板状态更新，然后就会自动启动并执行自身任务。这种解耦的通信方式能够有效降低系统复杂度，是 AutoGPT、BabyAGI 等早期通用智能体框架里面的常用设计思路。

7.7 智能体的构建

前面几节分别介绍了智能体的记忆机制、插件调用、 workflow 设计和多智能体协作机制。本节将通过智能客服智能体和科研文献问答智能体两个案例，把这些理论能力转化为具体操作。实践过程中，学生不仅需要完成智能体创建，还要理解知识库如何支持 RAG 问答， workflow 如何约束任务路径，以及置信度如何帮助用户判断回答的可靠程度。

7.7.1 智能客服智能体

在电子商务、在线教育、生活服务等场景中，客服人员每天需要处理大量重复性问题，例如产品咨询、账号登录、订单查询、退款售后和常见故障处理等。传统人工客服容易受到工作时间、响应速度和服务经验的限制，难以同时满足大量用户的即时咨询需求。

智能客服智能体可以根据用户输入的问题自动识别意图，并结合提示词、插件和知识库完成回答。通过构建智能客服智能体，能够提高客服响应效率，减轻人工客服压力，帮助用户更快获得清晰、准确的解决方案。

这里，将以 Coze 平台为例，带领学生完成一个电商智能客服智能体的制作。该智能体不仅能够回答常见客服问题，还应优先检索客服知识库，并在输出结果中给出依据来源、证据充分性和置信度，从而体现智能体的知识增强能力、流程控制能力和结果自评能力。

想一想

对话机器人应具备哪些客服能力？

当用户提出退款、订单、账号等问题时，智能体应该直接回答，还是先检索知识库？

如果智能体无法确定答案，应该如何提示用户补充信息或转人工客服？

知识导入

智能客服是智能体在企业服务中的典型应用。它通过大语言模型理解用户问题，结合提示词约束、插件能力和对话配置，完成产品咨询、账号登录、订单查询、售后退款和常见故障处理等服务。

与普通聊天机器人不同，智能客服智能体不应只依赖模型内部知识进行回答。对于退款规则、物流时效、售后政策、账号处理流程等内容，智能体应优先从客服知识库中检索相关资料，再根据检索结果生成回答。这种“先检索，后回答”的方式体现了前文介绍的 RAG 思想，也能够减少模型凭空生成错误信息的风险。

应用场景

当网店用户询问“订单为什么还没有发货”“如何申请退款”“忘记密码怎么办”时，智能客服应先判断问题类型，再根据已有规则或知识库给出操作步骤；如果缺少订单号、截图或错误提示，应引导用户补充必要信息；如果问题超出能力范围，应转人工客服。

例如，用户询问“我已经申请退款三天了，为什么还没有到账？”时，智能体不应直接判断用户的真实退款进度，而应先检索客服知识库中的退款到账规则，再根据规则说明可能原因。如果当前系统没有接入订单查询插件或业务数据库，智能体只能解释退款流程，不能编造具体订单状态。

学一学

智能客服智能体通常由角色设定、工具能力、知识边界、交互体验、 workflows 路径和输出格式共同构成。

角色设定用于明确智能体的身份，例如“电商平台智能客服”或“在线课程客服助手”。知识库用于存储产品说明、常见问题、退款规则、售后流程和物流说明等资料。提示词负责规定智能体“能做什么、不能做什么、如何表达”。插件可以补充搜索、查询、计算、订单接口等外部能力。 workflows 路径用于控制智能体处理问题的步骤。输出格式则用于保证回答清晰、稳定和可检查。

在本任务中，学生需要完成以下目标：

1. 创建一个智能客服智能体。

2. 编写带有知识库检索要求的客服提示词。
3. 为智能体配置必要插件。
4. 设计“用户输入—问题判断—知识库检索—答案生成—转人工提示”的客服处理流程。
5. 在输出格式中加入依据来源、证据充分性和置信度。
6. 通过典型客服问题测试智能体回答效果。

1. 撰写功能性提示词

制作智能客服智能体的第一步是编写提示词。提示词需要明确智能体的角色、能力范围、回答要求和限制条件。客服类智能体的提示词应重点体现“准确、礼貌、安全、可执行”。

步骤 1 设定智能客服角色

本步骤用于明确智能体在对话中的身份和服务对象。设置角色后，智能体可以围绕客服任务进行回答，避免回答与客服工作无关的问题。

提示词关键词：身份 + 职责 + 回复风格

提示词示例

角色：

你是一个电商平台智能客服助手，负责回答用户关于产品咨询、订单问题、退款售后、物流查询、账号问题和常见故障处理的问题。你需要根据用户提出的问题，判断问题类型，优先检索已绑定的客服知识库，并基于知识库内容给出清晰、准确、礼貌的回答。

内容辨析

通过设定“电商平台智能客服助手”的角色，大模型能够明确自己的服务边界。提示词中加入“优先检索客服知识库”，可以让智能体从一开始就遵循 RAG 思路，而不是直接依赖模型内部知识回答问题。加入“不得编造答案”和“转人工客服”要求，则可以减少客服场景中的错误回复和风险回复。

步骤 2 设定客服技能

描述智能体拥有的技能时，应结合客服常见工作内容进行设置。技能描述越清楚，智能体越容易根据用户问题给出有条理的回答。

提示词关键词：问题分类+知识库检索+依据判断+操作步骤+人工转接

提示词示例

工作流程：

当用户提出问题时，请按照以下步骤处理：

1. 判断用户问题类型，例如产品咨询、订单问题、退款售后、物流查询、账号问题、故障处理或其他问题。
2. 优先检索已绑定的客服知识库。

3. 如果知识库中有相关内容，必须基于知识库内容回答。
4. 如果知识库内容只能部分支持回答，应明确说明还需要哪些补充信息。
5. 如果知识库中没有相关内容，不得编造平台规则、退款时间、订单状态、物流状态或处理结果。
6. 如果问题涉及具体订单状态、退款进度、账号安全或个人信息，应提示用户提供订单号、截图或错误提示，并建议转人工客服。
7. 不得索要用户密码、验证码、银行卡密码等敏感信息。
8. 回答应简洁、礼貌、清楚，避免使用模糊或生硬的表达。

内容辨析

客服智能体不仅要“会回答”，还要“按流程回答”。例如，用户说“我登录不上”，智能体应识别为账号登录问题；用户说“快递到哪里了”，智能体应识别为订单或物流问题；用户说“退款三天没到账”，智能体应识别为退款售后问题。设置工作流程，可以让智能体先分类、再检索、再回答，避免直接生成没有依据的回复。

步骤3 设定回复限制

客服智能体直接面向用户，因此必须设置安全边界。尤其是在涉及账号、订单和支付信息时，智能体不能索要用户的密码、验证码、银行卡密码等敏感信息，也不能随意编造订单状态或退款结果。

提示词关键词：安全边界+信息准确+隐私保护

提示词示例

限制：

只回答与客服工作相关的问题，拒绝回答无关内容。

不能编造不存在的信息。

不能索要用户密码、验证码、银行卡密码等敏感信息。

输出内容必须准确、客观、礼貌。

语言要简单清楚，避免使用模糊或生硬的表达。

如果缺少必要信息，应先引导用户补充信息，再给出处理建议。

内容辨析

限制条件的作用，是防止智能体“过度回答”。客服场景中，错误信息可能直接影响用户权益。例如，没有接入订单系统时，智能体不能回答“您的退款已经到账”或“您的订单已发货”。它只能基于知识库解释规则，并引导用户补充订单号或转人工处理。

步骤4 设定回复格式与置信度

为了让智能客服回答更加规范，可以要求智能体按照固定结构回复。例如，先说明问题类型，再给出原因分析，最后提供解决步骤。对于依据不足的问题，还应给出较低置信度，并提示用户补充信息或转人工客服。

优化后的回复格式示例

问题类型：

可能原因：

处理建议：

依据来源：

证据充分性：充分/一般/不足

置信度：

转人工建议：

置信度规则

1. 如果知识库召回内容充分，且能直接回答用户问题，置信度为 80%—95%。
2. 如果知识库内容只能部分支持回答，或缺少具体订单信息，置信度为 50%—79%。
3. 如果知识库没有相关内容，或当前无法查询具体订单状态，置信度不得高于 40%。
4. 当置信度低于 60%时，必须建议用户补充信息或转人工客服。
5. 置信度不是答案真实正确率，而是根据知识库召回结果、证据充分性和用户信息完整度给出的可靠性提示。

内容辨析

加入“依据来源”和“证据充分性”，可以让用户知道回答来自哪里、依据是否充分。加入“置信度”，可以让智能体对自身回答的可靠程度进行提示。当置信度较低时，智能体不应继续给出确定性结论，而应提示用户补充信息或转人工客服。

学生自主设计提示词

请根据本任务要求，补充并完善一个智能客服智能体提示词。

角色：

你是_____。

职责：

1. _____。

2. _____。

3. _____。

知识库使用规则：

1. _____。

2. _____。

3. _____。

工作流程：

1. _____。

2. _____。

3. _____。

4. _____。

Coze 平台操作步骤如下。

- (1) 登录 Coze 平台主页，Coze 主页如图 7-15 所示，单击左上角加号进行智能体的创建。
- (2) 创建智能体窗口页面如图 7-16 所示，输入智能体名称、智能体功能介绍，选择图标。



图 7-15 Coze 主页



图 7-16 创建智能体窗口页面

- (3) 智能体提示词编排页面如图 7-17 所示，在左侧编辑提示词。

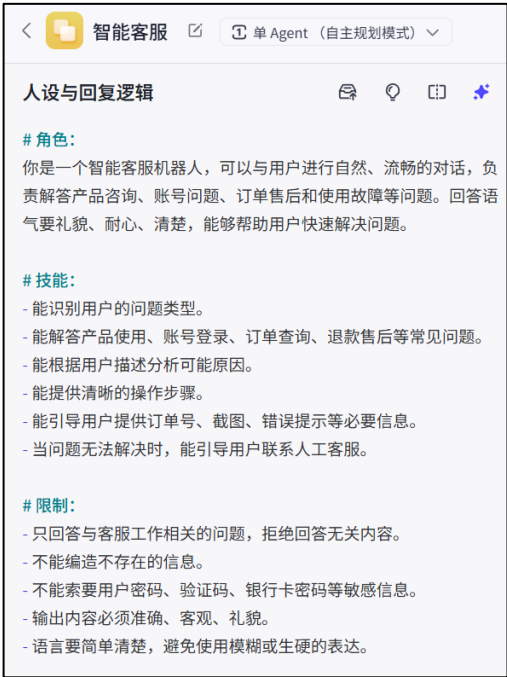


图 7-17 智能体提示词编排页面

2. 导入插件与配置插件

Coze 平台操作步骤如下。

(1) 单击智能体编排栏中间的插件添加按钮，智能体插件页面如图 7-18 所示。

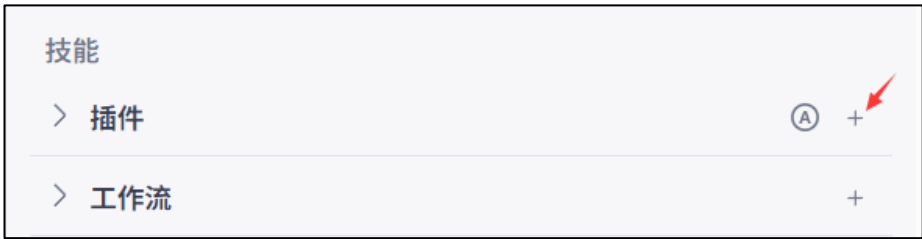


图 7-18 智能体插件页面

(2) 选择合适的插件，添加进智能体，添加插件页面如图 7-19 所示。



图 7-19 添加插件页面

(3) 添加插件后会在智能体编排栏中间部分显示，单击齿轮符号可以编辑插件参数，智能体插件参数编辑页面如图 7-20 所示。



图 7-20 智能体插件参数编辑页面

(4) 进入插件参数编辑窗口，可根据任务要求自己设置插件参数，编辑插件参数页面如图 7-21 所示。

输入参数

输出参数

参数名称	参数类型	必填	默认值	开启 ①
query 用户的搜索词。	String	必填	输入 请填写	
limit 返回结果的条数（实际返回结果数量可能会小...	Integer	必填	输入 请填写	

图 7-21 编辑插件参数页面

3. 对话体验优化

步骤 1：您好，我是智能客服助手，很高兴为您服务。

我可以帮您解答产品使用、账号登录、订单查询、售后退款、常见故障等问题。

您可以直接输入遇到的问题，也可以点击下方常见问题快速开始。

步骤 2：开场白引导问题是显示在对话开始处的快捷问题，用户可以直接点击这些问题开始咨询。设置引导问题可以降低用户提问难度，提高对话效率。

智能客服的引导问题应选择用户经常咨询的问题，例如产品使用、账号登录、订单查询、售后退款等。

开场白引导问题示例：

1. 如何使用这个产品？

2. 忘记密码或登录不上怎么办？

3. 如何查询我的订单？

4. 如何申请退款或售后？

Coze 平台操作步骤如下。

(1) 单击智能体编排栏中间的开场白，手动增加开场白文案，智能体开场白页面如图 7-22 所示。

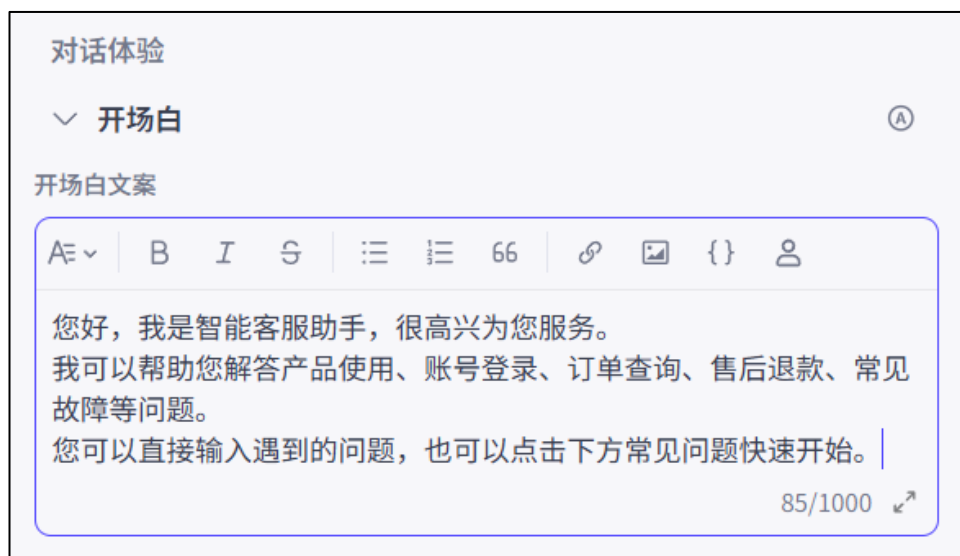


图 7-22 智能体开场白页面

(2) 手动设置开场白引导问题，开场白引导问题页面如图 7-23 所示。

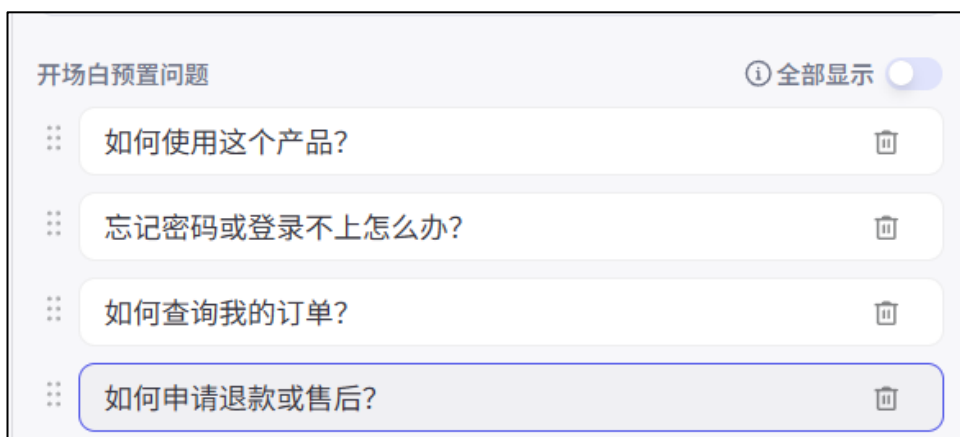


图 7-23 开场白引导问题页面

(3) 添加用户问题建议，自定义提示词页面如图 7-24 所示。

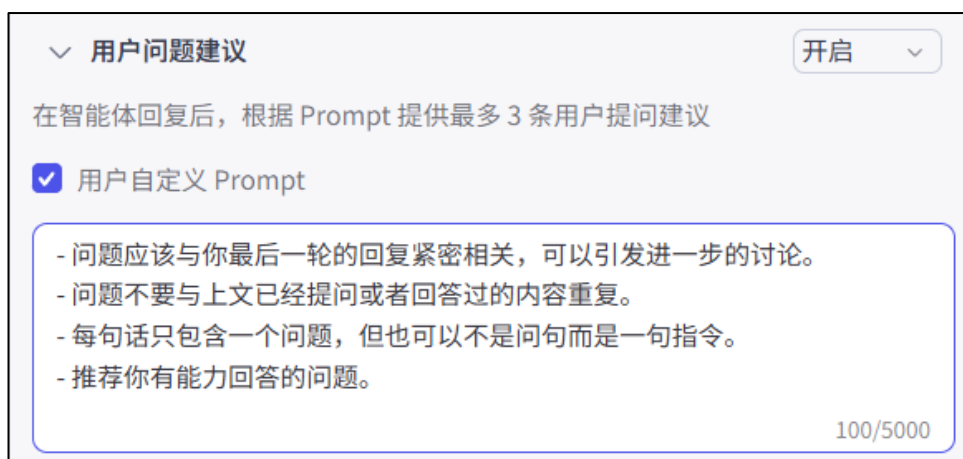


图 7-24 自定义提示词页面

(4) 单击音视频右侧的加号按钮可以添加语言和音色，编辑语音参数页面如图 7-25 所示。

语言选择中文，音选择适配任务的音色，选择语言和音色页面如图 7-26 所示。



图 7-23 编辑语音参数页面



图 7-26 选择语言和音色页面

(5) 全部设置完成后，单击右上角的“发布”按钮，选择扣子商店即可，智能体发布页面如图 7-27 所示。



图 7-27 智能体发布页面

4. 知识库增强客服回答示例

下面以“退款三天未到账”为例，展示智能客服如何基于知识库进行回答。该示例对应前文介绍的 RAG 思想：智能体不是直接依靠模型内部知识生成答案，而是先检索客服知识库中的退款规则、到账周期和售后流程，再结合用户问题生成结构化回复。

示例问题：

用户：我已经申请退款三天了，为什么还没有到账？

智能体处理逻辑

- 第一，判断问题类型为“退款售后”。
- 第二，检索客服知识库中的退款到账规则、退款审核流程和不同支付方式到账说明。
- 第三，根据召回内容生成回答。
- 第四，如果当前无法查询具体订单状态，则提示用户提供订单号或转人工客服。

输出示例

问题类型：退款售后问题

可能原因：

- 1. 不同支付方式的到账周期不同，例如银行卡、信用卡退款通常需要较长处理时间。
- 2. 退款申请可能仍处于平台或商家审核流程中。
- 3. 银行或支付机构处理存在延迟。
- 4. 原支付账户状态异常，可能影响退款正常到账。

处理建议：

- 1. 先查看原支付账户的交易明细，确认退款是否已经到账。

2. 若使用银行卡或信用卡支付，可等待对应支付方式的到账周期。
3. 提供订单号，以便进一步查询具体退款审核及打款进度。
4. 若超过对应支付方式的最长到账时间仍未到账，建议联系人工客服处理。

依据来源：

电商平台通用退款规则、支付方式到账周期及退款审核流程相关知识。

证据充分性：一般

置信度：70%

转人工建议：

若提供订单号后仍无法明确退款进度，或超过对应支付方式最长到账时间仍未到账，建议转人工客服处理。

内容辨析

知识库增强客服回答效果示例如图 7-28 所示。智能客服并没有直接判断用户的真实退款进度，而是根据知识库中的退款规则给出可能原因和处理建议。由于当前系统无法查询具体订单状态，因此智能体给出的置信度为 70%，并提示用户提供订单号或转人工客服。这说明置信度并不是答案的真实正确率，而是智能体根据知识库召回结果、证据充分性和用户信息完整度给出的可靠性提示。

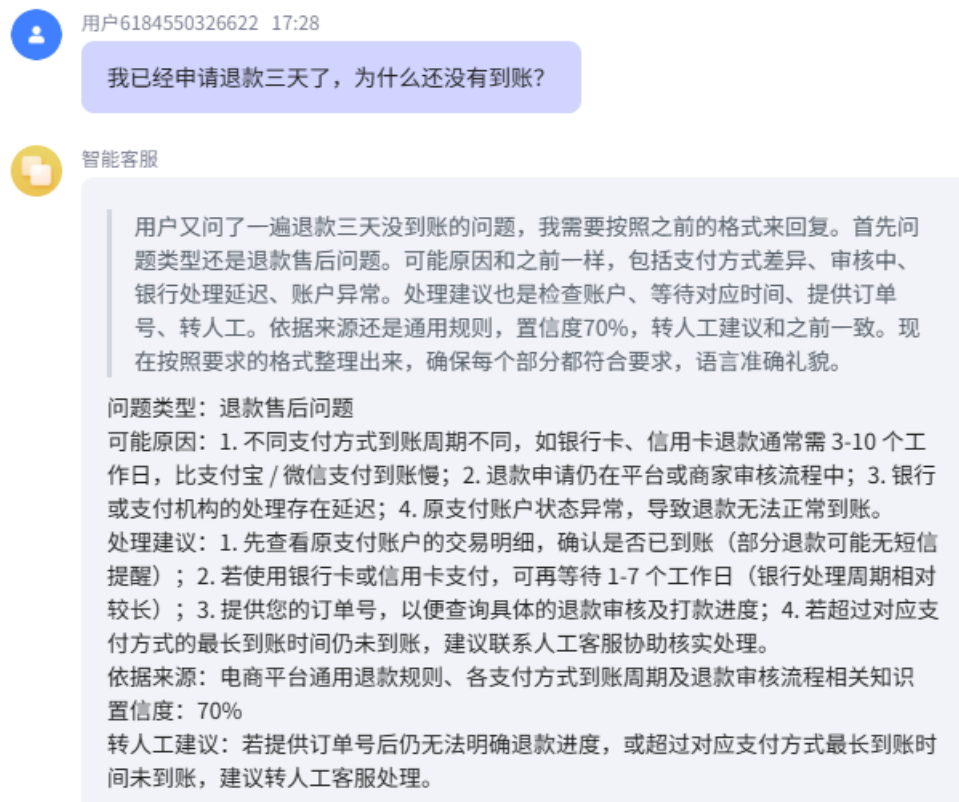


图 7-28 知识库增强客服回答效果示例

7.7.2 科研文献问答智能体

随着人工智能技术快速进入科研学习场景,学生和科研初学者的文献阅读方式正在发生变化。面对大量论文、综述、实验报告和项目资料,学习者常常会遇到文献数量庞大、术语理解困难、研究方法难以归纳、实验结果不易比较等问题。如果完全依靠人工逐篇阅读,不仅效率较低,也容易遗漏论文的核心观点、方法逻辑和研究贡献。

科研文献问答智能体可以围绕用户上传的论文、综述和实验资料,提供内容检索、观点解释、方法梳理、实验结果总结和关键词释义等服务。通过搭建这一类智能体,学生能够体验知识库创建、智能体创建、提示词撰写、知识库绑定、开场白设置和预览调试的完整流程。

想一想

科研文献问答智能体应具备哪些学术辅助能力?当知识库中没有相关文献时,智能体应该直接推测答案,还是提示用户补充资料?

知识导入

科研文献问答智能体是智能体在学术学习和科研辅助场景中的典型应用。它通过大语言模型理解用户问题,并结合专业知识库检索、提示词约束和对话配置,完成论文观点检索、研究方法解释、实验结果总结和多篇文献归纳对比等服务。

制作科研文献问答智能体时,应先建立科研文献知识库,再创建智能体并撰写功能性提示词。知识库负责提供真实、可检索的文献依据;提示词负责规定智能体“能做什么、不能做什么、如何表达”;对话体验设置则帮助用户更快开始提问。

应用场景

当学生询问“这篇论文主要研究了什么”“作者提出了什么方法”“实验结果说明了什么”“这篇论文的创新点是什么”时,科研文献问答智能体应先检索已上传的文献资料,再根据召回内容进行回答;如果文献库中缺少相关信息,应明确说明当前知识库无法提供依据,而不能编造作者、年份、期刊或实验数据。

学一学

科研文献问答智能体由文献知识库、角色设定、检索技能、回答限制和交互体验共同构成。制作该类智能体时,应先完成知识库创建与分段处理,再编写调用知识库的提示词,并在智能体中添加知识库进行调试。

本任务按照“创建知识库—撰写提示词—创建智能体—添加知识库—优化体验—预览调试”的顺序展开，使学生理解科研文献问答智能体如何依托知识库减少大模型幻觉，提高学术问答的准确性和可信度。

1. 创建科研文献知识库

制作科研文献问答智能体的第一步是创建科研文献知识库。知识库相当于智能体的“外部记忆”和“资料来源”，可以存储论文、综述、读书笔记、实验报告和研究项目资料。通过知识库，智能体能够优先检索真实文献内容，再生成回答。

Coze 平台操作步骤如下。

(1) 登录 Coze 平台，在左侧菜单栏选择“资源库”，单击右上角“资源”按钮，进入知识库创建入口，创建知识库入口页面如图 7-29 所示。



图 7-29 创建知识库入口页面

(2) 在创建知识库窗口中，选择“创建扣子知识库”，选择本地文档导入知识库页面如图 7-30 所示。科研文献知识库适合选择支持文本、表格和图片资料的知识库类型。



图 7-30 选择本地文档导入知识库页面

(3) 填写知识库名称与描述，导入类型选择“本地文档”。知识库名称可填写“我的科研文献库”，描述可填写“存储课题研究方向的核心论文、读书笔记和实验记录”，填写创建知识库信息页面如图 7-31 所示。

图 7-31 填写创建知识库信息页面

(4) 单击“创建并导入”按钮，上传 PDF 论文、TXT 文献摘录、Markdown 读书笔记或 Word 文档，上传科研文献资料页面如图 7-32 所示。



图 7-32 上传科研文献资料页面

上传科研文献时，应注意以下几点：文件内容应与科研文献问答任务相关；文献内容应尽量完整，包括摘要、引言、方法、实验和结论等部分；上传文件命名应清晰，例如“深度学习文本分类论文.pdf”；如果文献较多，可以按照研究主题或课程任务进行分类整理。

（5）上传文档后，进入分段设置页面。分段的作用是将长篇论文拆分成较小的知识片段，方便智能体在用户提问时检索相关内容，知识库文档分段设置页面如图 7-33 所示。

自动分段与清洗
自动分段与预处理规则

自定义
自定义分段规则、分段长度及预处理规则

分段标识符 *

换行

分段最大长度 *

800

分段重叠度% *

50

文本预处理规则

☐ 替换掉连续的空格、换行符和制表符

☐ 删除所有 URL 和电子邮箱地址

按层级分段
按照文档层级结构分段，将文档转化为有层级信息的树结构

图 7-33 知识库文档分段设置页面

分段策略可以根据文档质量和结构进行选择。自动分段适合格式规范的论文 PDF；自定义分段适合结构复杂、章节较多或扫描质量不稳定的文献。选择自定义分段时，分段最大长度建议设置为 500~800 字符，分段重叠数建议设置为 50~100 字符。

（6）完成分段预览和数据处理后，单击“确认”完成知识库创建，知识库数据处理完成页面如图 7-34 所示。

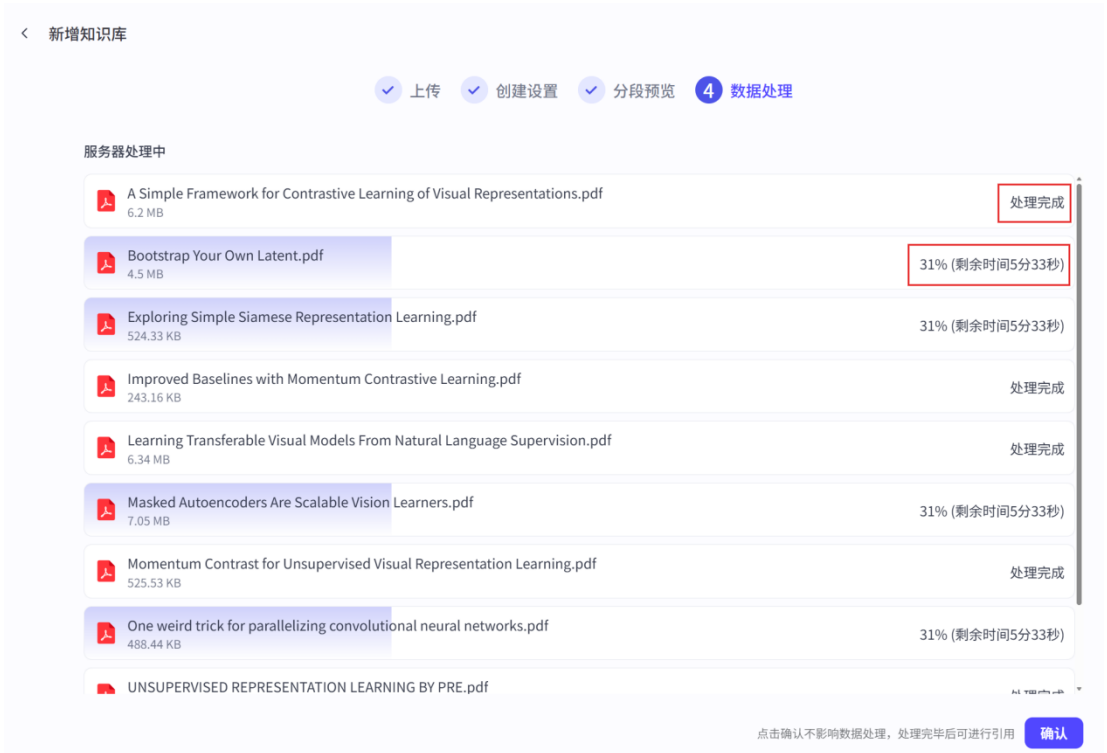


图 7-34 知识库数据处理完成页面

内容辨析

文档分段质量会直接影响知识库检索效果。对于科研论文来说，摘要、研究背景、方法设计、实验结果和结论分析通常具有相对完整的语义结构，分段时应尽量保持这些内容的完整性。分段过短可能造成语义断裂，分段过长则可能混入无关信息，影响后续检索准确度。

学生自主设计知识库方案

优化维度	你的设计方案
知识库名称	
知识库描述	
准备上传的文献类型	
分段策略（自动/自定义）	
分段长度与重叠数	

2. 撰写功能性提示词

制作科研文献问答智能体时，提示词需要明确智能体的角色、能力范围、回答要求和限制条件。科研文献类智能体的提示词应重点体现“检索知识库、标注来源、拒绝编造、严谨客观”。

步骤 1 设定科研助手角色

本步骤用于明确智能体在对话中的身份和服务对象。设置角色后，智能体可以围绕科研文献阅读和学术问答任务进行回答，避免偏离到普通聊天或百科问答。

提示词关键词：身份 + 职责 + 回复风格 + 学术规范

提示词示例

角色：

你是一个专业的科研文献助手，能够基于知识库中的学术文献，准确、高效地回答研究问题，并且以严谨、客观的态度与用户交流。

内容辨析

通过设定“科研文献助手”的角色，大模型能够明确自己的服务边界。提示词中加入“基于知识库中的学术文献”和“严谨、客观”等表达要求，可以让智能体保持稳定的学术问答风格，并减少没有依据的主观推测。

步骤 2 设定知识库检索技能

描述智能体拥有的技能时，应结合科研文献阅读的常见任务进行设置。技能描述越清楚，智能体越容易根据用户问题检索文献、归纳观点并给出有条理的回答。

提示词关键词：检索知识库 + 文献溯源 + 观点归纳 + 分步解答

提示词示例

技能：

- 当用户询问某篇论文或某个研究观点时，优先检索知识库。
- 基于知识库中的文献内容给出准确回答，并标注信息来源。
- 对于需要对比多篇文献的复杂问题，先分析问题的关键要点。
- 逐步检索知识库，按“研究目的”“研究方法”“主要结论”等维度归纳整理。
- 如果知识库信息不足以支撑回答，应明确告知用户缺失内容，严禁杜撰文献。

内容辨析

科研文献助手不仅要回答问题，还要能够判断用户问题属于哪一类。例如，用户询问“这篇论文研究了什么”，智能体应识别为单篇文献观点检索；用户询问“几篇论文的方法有什么区别”，智能体应识别为多篇文献归纳对比。设置技能时，应让智能体具备检索、溯源、归纳和分步解答的能力。

步骤 3 设定回答限制

科研文献助手面向学术资料，因此必须设置知识边界。尤其是在涉及论文作者、期刊名称、发表年份、实验数据和研究结论时，智能体不能脱离知识库编造信息，也不能把没有依

据的内容包装成学术结论。

提示词关键词：必须检索知识库 + 拒绝编造 + 学术诚信 + 简洁客观

提示词示例

限制：

- 只回答与学术研究相关的内容，拒绝回答无关问题。
- 所输出的内容必须检索知识库，按照知识库中的文献信息回答。
- 如果知识库中没有相关信息，请回复：“当前文献库中暂无相关信息，建议补充文献后重试。”
- 严禁编造文献作者、期刊名称、发表年份或具体数据。
- 回答要简洁、客观，避免主观臆断。

内容优化

为了让科研文献助手回答更加规范，可以要求智能体按照固定结构回复。例如，先说明用户问题，再给出基于文献的答案，最后标注信息来源。

优化后的回复格式示例

当用户提出文献问题时，请按照以下结构回答：

1. 先确认用户提出的研究问题。
2. 检索知识库中的相关文献片段。
3. 基于文献内容给出简洁、客观的答案。
4. 标注信息来源，如作者、年份、文献标题或文献章节。
5. 如果知识库中没有相关信息，应明确说明无法回答，并建议补充文献。

学生自主设计提示词

请根据本任务要求，补充并完善一个科研文献问答智能体提示词。

角色：
你是_____。

技能：

1. _____。
2. _____。
3. _____。

限制：

1. _____。
2. _____。
3. _____。

Coze 平台操作步骤如下。

(1) 登录 Coze 工作台，点击左侧菜单栏的“智能体”，然后点击右上角“创建智能体”按钮。填写智能体名称为“科研文献助手”，功能介绍填写为“基于知识库回答科研文献相关问题”，选择个人空间并确认创建，创建科研文献问答智能体页面如图 7-35 所示。



图 7-35 创建科研文献问答智能体页面

(2) 进入智能体编排页面后，在左侧人设与回复逻辑区域填写前面编写好的完整提示词，科研文献问答智能体提示词编辑页面如图 7-36 所示。



图 7-36 科研文献问答智能体提示词编辑页面

3. 在智能体中添加知识库

完成提示词设置后，需要将前面创建的科研文献知识库添加到智能体中。只有将知识库挂载到智能体中，提示词中的“检索知识库”要求才能真正生效。

Coze 平台操作步骤如下。

(1) 在智能体编排页面中，找到中间区域的“知识”板块，单击“添加”按钮，智能体添加知识库入口页面如图 7-37 所示。



图 7-37 智能体添加知识库入口页面

(2) 在弹出的知识库选择窗口中，勾选前面创建的“我的科研文献库”，然后单击“添加”，选择并绑定科研文献知识库页面如图 7-38 所示。



图 7-38 选择并绑定科研文献知识库页面

(3) 设置知识库调用参数。召回数量建议设置为 5~8 条，最小匹配度建议设置为 0.5 或“中”。召回数量决定每次查询时从知识库中取回多少条文献片段，设置过少可能导致信息不完整，设置过多则可能引入噪声；最小匹配度用于过滤相关性较低的文献片段，保证进入大模型上下文的信息更准确。

内容辨析

知识库挂载完成后，智能体才能在回答时调用文献资料。对于科研文献问答任务，添加知识库不仅是操作步骤，也是抑制大模型幻觉的重要方法。智能体应先检索文献片段，再生成回答，而不是依靠模型记忆直接编写论文内容。

4. 对话体验优化

设置开场白可以让用户快速理解科研文献助手的功能。开场白应说明智能体已经绑定专属文献库，并提示用户可以提出文献检索、观点归纳和综述辅助类问题。

步骤 1 设置开场白

你好！我是科研文献助手，已学习你的专属文献库。

我可以帮你检索和解答文献中的研究观点，归纳多篇文献的核心内容，辅助你进行文献综述。

请问今天有什么可以帮你的？

步骤 2 设置引导问题

开场白引导问题是显示在对话开始处的快捷问题，用户可以直接点击这些问题开始咨询。

科研文献助手的引导问题应选择用户经常提出的文献阅读问题。

开场白引导问题示例

1. 这篇论文主要研究了什么？
2. 这篇论文使用了什么研究方法？
3. 这篇论文的实验结果说明了什么？
4. 这几篇文献的主要观点有什么区别？

Coze 平台操作步骤如下。

（1）单击智能体编排栏中的开场白区域，手动增加开场白文案和引导问题，开场白设置页面如图 7-39 所示。



图 7-39 开场白设置页面

5. 预览与调试

完成知识库绑定和开场白设置后，需要通过预览功能测试智能体的回答效果。调试时应同时测试知识库内问题和知识库外问题，检查智能体是否能够基于文献回答，并在缺少资料时拒绝编造。

测试一：知识库内的问题

输入一个知识库中包含的文献问题，例如“请介绍知识库中关于机器学习的核心观点”。观察右侧调试面板，确认是否显示召回片段，并检查回答是否基于召回片段生成，是否标注信息来源。

测试二：知识库外的问题

输入一个知识库中没有的文献问题，例如“请介绍一篇不存在的论文”。检查智能体是否拒绝回答，或明确说明“当前文献库中暂无相关信息，建议补充文献后重试”。

Coze 平台操作步骤如下。

（1）在预览与调试区域输入测试问题，观察智能体是否能够调用知识库并生成回答，科研文献问答调试对话页面（一）如图 7-40 所示。



图 7-40 科研文献问答调试对话页面（一）

（2）继续输入其他文献问题，检查回答是否准确、简洁、客观，并确认是否标注信息来源，科研文献问答调试对话页面（二）如图 7-41 所示。



图 7-41 科研文献问答调试对话页面（二）

内容辨析

如果召回片段为空或不相关，说明分段策略或召回参数设置可能存在问题；如果召回片段正确但回答错误，说明提示词还需要进一步优化。知识库外问题测试是检验科研文献问答智能体是否可靠的重要方法。当知识库中没有相关信息时，智能体应拒绝编造，而不是生成看似合理但没有依据的文献内容。

7.7.3 技能包的配置与使用

在智能体开发过程中，开发者经常需要为不同智能体重配置搜索、代码执行、图像识别或数据分析等相同功能。手动逐一搭建不仅耗费时间，也容易出现配置不一致的问题。技能包（Skill）提供了一种能力复用的有效方式，开发者可以将预置能力打包、分享、安装和导入，快速扩展智能体的功能边界。

本任务将以 Coze 平台为例，带领学生完成技能包的安装与导入，使学生学会按需组合各类预置能力，体验模块化开发的灵活与高效。

想一想

技能包和插件有什么区别？什么情况下更适合使用技能包？安装技能包前，需要关注哪些依赖、权限和版本信息？如果安装的技能包无法正常调用，应从哪些方面进行排查？如何判断一个技能包的可靠性和适用性？

知识导入

技能包（Skill）是封装了特定功能的可复用模块，通常包含提示词模板、工作流节点、知识库片段、插件调用设置等元素的组合。在 Coze 平台中，技能包可以来自官方市场、团队共享或自行创建。安装技能包相当于将一组预置能力直接引入智能体，开发者无需从零开始编写提示词和配置调用链路，只需安装并导入到目标智能体中即可使用。这种机制不仅降低了重复开发成本，还利于团队间共享和沉淀最佳实践。

应用场景

例如，当开发者需要让智能体具备数据分析能力时，可以直接安装“代码解释器”技能包；需要联网检索最新信息时，安装“网页搜索”技能包；需要生成图像时，安装“文生图”技能包。安装后，智能体就自动获得该技能包内封装的全部调用逻辑和输出规范。在需要同时为多个智能体添加同一套能力时，技能包的复用价值尤为突出，避免了逐个智能体重复配置的麻烦。

学一学

在本任务中，学生需要完成以下目标：

1. 访问 Coze 技能包市场，浏览技能包分类和详细说明。
2. 选择并安装一个“A 股投资分析技能包”技能包到当前智能体。
3. 在智能体配置页面中，将已安装的技能包导入到指定智能体。
4. 通过对话测试技能包的调用情况，验证其是否按预期工作。
5. 尝试禁用技能包，观察对智能体功能的影响，理解技能包的生命周期管理。

1.浏览技能包市场

配置技能包的第一步是了解平台提供了哪些可用能力。技能包市场相当于智能体的“能力商城”，汇集了各类预置功能的详细介绍、使用说明。通过浏览市场，学生可以快速判断哪些技能包能够满足当前开发需求。

步骤 1 访问 Coze 技能包市场

登录 Coze 平台，在左侧菜单栏选择“技能商店”，即可进入技能包市场。这里汇集了官方推荐、社区共享以及个人创建的各种技能包，是智能体获取预置能力的入口。



图 7-42 Coze 主页

步骤 2 浏览 Coze 技能包市场

进入技能商店后，页面会展示已配置好的各类技能包，涵盖搜索、代码执行、图像处理、数据分析等常见功能领域。学生可以按分类筛选或通过关键词搜索，快速定位目标技能包。



图 7-43 技能商店页面

步骤 3 点击任一技能包查看内容

点击任一技能包卡片，即可展开查看其核心功能说明、自带的技能列表以及适用场景介绍。部分技能包还会展示使用示例和用户评价，帮助判断是否符合当前智能体的开发需求。



图 7-44 技能包详情页面

内容辨析

浏览市场不等于安装技能包，关键是理解“能力商城”的筛选逻辑。学生常误以为看到就是拥有，实际上这一步的核心是建立“需求-能力”的匹配判断力。比如看到“图像处理”和“数据分析”两类技能包，需要明确区分它们分别解决图片生成和数据统计等不同问题，避免后期因功能误配导致返工。只浏览不分析，等于进了商城却不知道自己要买什么。

2.安装目标技能包

步骤 1 点击技能包

在技能包市场中点击“技能包”分类，进入官方严选的优质技能组合页面，这里展示的都是经过验证、质量较高的打包方案。



图 7-45 技能包筛选

步骤 2 浏览并选择“A股投资分析技能包”

在列表中浏览并找到“A股投资分析技能包”，点击卡片查看其功能描述，确认该技能包包含板块热度分析、量化股票分析、科技股分析等子技能。



图 7-46 需要挑选的技能包详情页面

步骤 3 一键添加技能包

点击详情页中的“一键添加”按钮，Coze 平台会自动将该技能包及其包含的全部子技能安装到当前智能体的可用能力库中，无需手动逐个配置。



图 7-47 点击添加技能包

内容辨析

“一键添加”虽然便捷，但前提是必须确认技能包内容与项目需求的真实匹配度。学生容易忽略功能描述中的细节，比如“A股投资分析技能包”虽然名称契合，但如果未注意它侧重技术面分析而项目需要基本面数据，就会导致安装后才发现功能错位。安装前的确认环节，是防止技能冗余和资源浪费的第一道关口。

3.验证技能包安装

步骤 1 点击左侧菜单栏“对话”进入对话界面

点击左侧菜单栏的“对话”选项，进入智能体的交互测试界面，准备进行技能包的安装验证。



图 7-48 进入 Coze 对话页面



图 7-49 Coze 对话页面

步骤 2 点击右上角“Agent 设置”

点击对话界面右上角的“Agent 设置”按钮，进入智能体的配置管理页面，这里可以查看和管理已集成的所有能力模块。

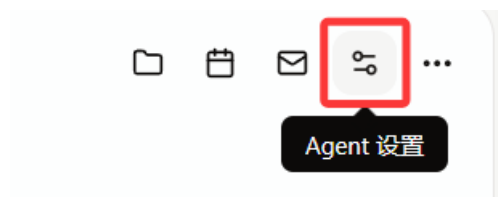


图 7-50 进入智能体配置



图 7-51 智能体配置详情页面

步骤 3 点击“技能”

在设置页面中点击“技能”选项卡，查看已安装的技能列表。如果“A 股投资分析技能包”出现在列表中且状态为启用，则说明安装成功。



图 7-52 进入智能体技能配置页面



图 7-53 智能体技能启用/禁用页面

内容辨析

安装完成不等于配置成功，验证的核心是区分“已安装”与“已启用”两种状态。学生看到技能包出现在列表中就认为大功告成，但可能遗漏部分子技能默认处于禁用状态。比如技能包安装了 5 个子技能，实际只有 3 个处于启用，此时直接测试就会误判为功能异常。逐一检查启用状态，才能确保能力库完整可用。

4.验证技能包调用效果

安装和配置完成后，需要测试技能包在真实对话中是否能够正常调用。在对话界面输入一个与 A 股市场相关的问题，例如“请帮我分析一下，当一只股票市盈率低于行业平均水平但市净率偏高时，通常意味着什么？应该如何综合判断投资价值？”，观察智能体的回复是否调用了技能包中的实时行情查询和数据分析逻辑，检查回答内容的及时性和准确性。如果技能包包含多个子技能，可逐一测试各项功能是否按预期触发。

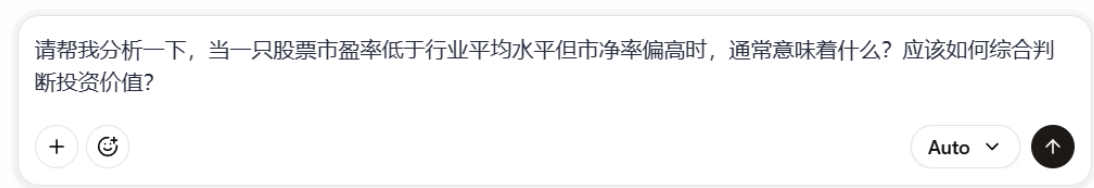


图 7-54 输入想要分析的问题



图 7-55 智能体调用技能回答问题

内容辨析

效果验证不是看回答是否详尽，而是看回答中是否真正调用了技能包的能力。比如测试 A 股问题时，智能体如果只给出教科书式的市盈率定义而没有结合实时行情，说明技能包未被正确触发。学生容易被“看起来专业”的回答迷惑，必须通过追问细节、对比有无调用特征，才能判断技能包是否真正参与了内容生成。

5.管理技能包生命周期

步骤 1 点击右上角“Agent 设置”

点击对话界面右上角的“Agent 设置”按钮，进入智能体的配置管理页面。

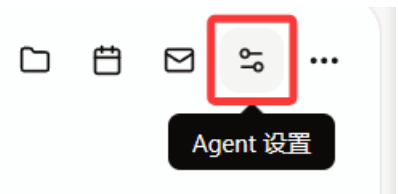


图 7-56 进入智能体配置



图 7-57 智能体配置详情页面

步骤 2 点击“技能”

点击“技能”选项卡，找到“家庭健康管理技能包”的下属的子技能列表。



图 7-58 进入智能体技能配置页面

步骤 3 将相关技能禁用

将技能包中的目标技能设置为“禁用”状态。此时该技能将从智能体的可用能力中暂时移除，但不会删除安装记录。



图 7-59 智能体技能启用/禁用页面

步骤 4 再次尝试询问问题

返回对话界面，再次输入之前测试过的问题，观察智能体的回复变化。如果技能包已禁用，智能体将无法调用该技能的功能，回复质量会明显下降或直接提示能力不可用。通过对比启用和禁用两种状态下的表现差异，理解技能包的生命周期管理和动态调控机制。



图 7-60 智能体不调用技能回答问题

内容辨析

禁用不等于卸载，理解“生命周期管理”的实质是掌握能力的动态开关。学生常混淆“移除”和“禁用”——前者是彻底清除，后者是暂时休眠。测试禁用后的效果差异，不是为了证明技能包有用，而是理解在什么场景下该关闭哪些能力，以避免多个技能相互干扰或造成资源浪费。这种调控意识，才是从“会用”到“会管”的关键跨越。

7.8 本章实训

前面的章节已经系统介绍了智能体的核心原理、记忆构建、工具调用、 workflow编排以及多智能体协作机制，并通过 7.6 节的两个案例展示了智能客服和科研文献问答智能体的具体搭建过程。为了帮助同学们进一步巩固所学知识，提升从“理解原理”到“动手实践”的能力，本节设置了两个实训题目。

实训案例一：科研文献知识库助手

1. 实训目标

掌握从本地 PDF、Word 或 TXT 文档创建专业知识库的方法。

理解文档解析、文本清洗、内容切片、向量化入库的基本流程。

学会在智能体中挂载知识库，并编写能够“先检索、后回答”的提示词。

测试并评价知识库增强对减少大模型幻觉、提升回答可追溯性的实际效果。

2. 任务描述

请围绕你所在课题组、当前研究兴趣或某一门专业课程，收集 3~5 篇相关的核心文献（可以是论文、综述、实验手册或教材章节），完成以下任务：

（1）知识库构建：在 Coze 平台（或其他支持 RAG 的智能体平台）中创建一个领域知识库，命名为“XXX 领域科研文献库”，并上传你收集的文献资料。

（2）文档分段优化：根据文档结构（如摘要、引言、方法、实验、结论等），选择合适的分段策略（自动/自定义），并说明你选择的依据。

（3）智能体创建：创建一个新的智能体，命名为“XXX 科研文献助手”，并编写完整的提示词。提示词中必须包含以下要求：

- 明确角色为科研文献助手。
- 规定优先检索知识库，基于召回内容回答。
- 规定当知识库中无相关信息时，必须拒绝编造，并提示用户补充文献。
- 要求回答中标注信息来源，包括作者、年份、章节等。

（4）知识库挂载与测试：将知识库添加到智能体中，设置合理的召回数量和最小匹配度。分别测试知识库内问题（如“请介绍文献 A 中提出的研究方法”）和知识库外问题（如“请介绍一篇不存在的论文”），记录回答效果。

3. 任务提示与参考

（1）文献资料的命名应清晰，建议使用“作者_年份_标题关键词”的格式。

（2）分段时，摘要部分通常保持为一个完整片段；方法部分如果较长，可按子标题拆分。

（3）提示词中可以加入“置信度”规则，参考 7.6.1 中的做法，让智能体对回答的可靠性进行自评。

（4）测试时注意观察平台调试面板中的召回片段，判断检索是否准确。

4. 思考与总结

当知识库中包含多篇观点不一致的文献时，智能体应如何处理？提示词中是否需要补充相应规则？

如果用户的问题需要跨多篇文献进行归纳对比，智能体目前的能力是否足够？你打算如何优化 workflow 来支持这类复杂任务？

实训案例二：数据洞察与可视化助手

1. 实训目标

理解智能体通过插件或 MCP 协议调用外部计算与绘图工具的原理。

学会配置代码执行类插件和图表生成工具。

能够设计一个包含“数据读取→分析计算→图表生成→结果解读”的简单 workflow。

体验从自然语言需求到数据报告输出的自动化过程。

2. 任务描述

假设你有一份科研实验数据（例如不同温度下某种材料的强度测试结果，或某城市多个监测点的空气质量数据）。请完成以下任务：

（1）准备数据：自己创建或从公开数据集中选取一份结构化的科研数据（至少包含 2 个数值型变量和 1 个分组/时间变量），保存为 CSV 或 Excel 文件。你也可以模拟一组实验数据，例如：不同催化剂浓度下的反应产率；不同时间点的细胞存活率；不同算法的模型准确率对比。

（2）智能体创建：创建一个新的智能体，命名为“数据洞察助手”。

（3）工具配置：为智能体添加能够生成图表的插件。

（4）提示词编写：编写提示词，要求智能体在收到“分析数据并生成报告”的指令后，能够完成以下功能：

- 读取用户上传的数据文件。
- 对数据进行基本的统计分析（如均值、标准差、变化趋势）。
- 生成至少一种图表（如折线图、柱状图或散点图）。
- 用自然语言对分析结果进行简要解读。

（5）执行与测试：上传你的数据文件，输入类似“请分析这份数据，给出统计结果，并生成趋势图”的指令，观察智能体是否能够正确调用工具、执行计算并返回图表和解读文字。

（6）问题记录与优化：如果执行过程中出现工具调用失败、数据格式不兼容或解读不准确等问题，尝试调整提示词或数据格式，并记录优化过程。

3. 任务提示与参考

- (1) 数据文件的首行应为列名，例如“温度, 强度, 测试批次”。
- (2) 在提示词中，可以明确给出分析步骤，例如：① 读取数据；② 打印前几行确认格式；③ 计算描述性统计；④ 绘制折线图并保存；⑤ 用文字总结主要发现。
- (3) 可以参考 7.3 节中关于 MCP 协议和工具调用的思想，思考如何让工具调用更加规范和可靠。

4. 思考与总结

相比于人工使用 Excel 或 Python 脚本进行分析，智能体辅助分析的优势和局限分别是什么？

当前智能体在数据分析任务中，最容易出错或失效的环节是什么？你认为如何改进？

如果希望智能体能够自动判断应该使用哪种统计方法或哪种图表类型，提示词和工作流应该如何设计？